# CRYPTO 2 - Lecture 4
# Multi-Party Computation in the Head

**Damien Vergnaud**

Sorbonne Université – CNRS

# Picnic Signatures: Overview

- consider some one-way function $F$
- picks uniformly at random $\mathrm{sk}$ in $F$'s domain
- sets and publishes $\mathrm{pk} = F(\mathrm{sk})$

- for identification, Alice proves (in **zero-knowledge**) that she knows $\mathrm{sk}$ such that $\mathrm{pk} = F(\mathrm{sk})$

- for signatures, Alice proves
  - in zero-knowledge
  - **non-interactively** (using Fiat-Shamir heuristic with the message)

  that she knows $\mathrm{sk}$ such that $\mathrm{pk} = F(\mathrm{sk})$

- for **post-quantum** security, $F$ has to be one-way against quantum computers

# Picnic Signatures: Overview

- $F : \{0,1\}^n \times \{0,1\}^n \to \{0,1\}^n$ a block cipher
  $\mathrm{sk} \in \{0,1\}^n$ is a random secret-key for $F$
  $\mathrm{pk} = F_{\mathrm{sk}}(m)$ where $m$ is a public (random) message in $\{0,1\}^n$

- classically, recovering $\mathrm{sk}$ given $\mathrm{pk}$ and $m$ requires $O(2^n)$ time
- quantumly, recovering $\mathrm{sk}$ given $\mathrm{pk}$ and $m$ requires $O(2^{n/2})$ time (Grover's algorithm)

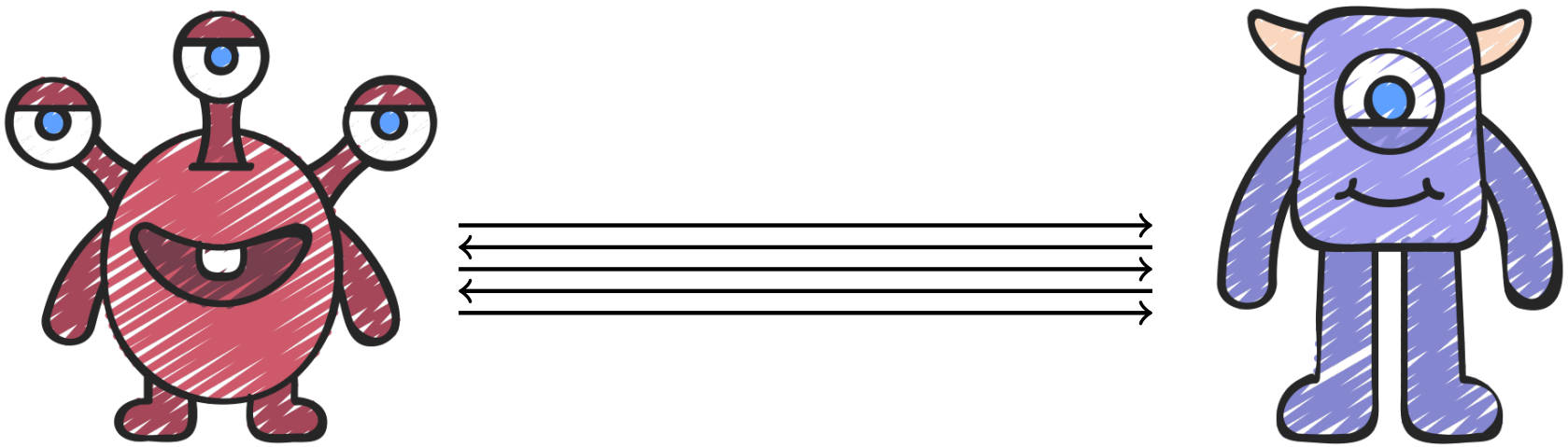
- many possible block ciphers (AES ?)
- needs an efficient ZK proof system to prove $\mathrm{pk} = F(\mathrm{sk})$
  - many efficient constructions based on discrete logarithms
    $\rightsquigarrow$ not **post-quantum**
  - many constructions based on generic assumptions
    $\rightsquigarrow$ inefficient

# Outline


1. Picnic Signatures - Overview
2. Zero-Knowledge Proofs and MPC
   - Zero-knowledge (recall)
   - MPC (recall)
3. MPC-in-the-Head
   - ZK from 2-private MPC
   - ZK from 1-private MPC
   - ZK from $n$-private MPC

# Zero-knowledge interactive proof

- interactive method to prove that a statement is true
  without revealing anything other than the veracity of the statement



Shafi Goldwasser, Silvio Micali, Charles Rackoff
The Knowledge Complexity of Interactive Proof-Systems
STOC 1985: 291-304 **(1993 Gödel Prize)**

# Zero-knowledge interactive proof

1. **Completeness:** if the statement is true, the honest verifier (that is, one following the protocol properly) will be convinced of this fact by an honest prover.
2. **Soundness:** if the statement is false, no cheating prover can convince the honest verifier that it is true, except with some small probability.
3. **Zero-knowledge:** if the statement is true, no cheating verifier learns anything other than this fact.

# Formalizing ZK

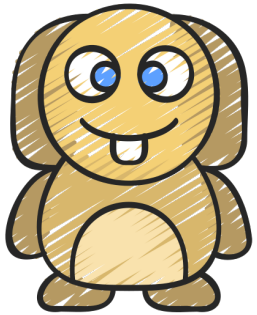
- the verifier $V$ has to be **efficient** (= PPTM)
  otherwise the definition is vacuous (prover is useless)
- *a priori* the prover $P$ is unbounded (but **useless** in practice)
  $\rightsquigarrow$ bounded prover (= PPTM) with additionnal information
- We consider $\mathcal{NP}$ languages: $\mathcal{L} = \{x | \exists w, \mathcal{R}(x, w) = 1\}$
  - $x$ is known to $P$ and $V$;
  - $P$ wants to convince $V$ that $x \in \mathcal{L}$;
  - $P$ knows a witness $w$ s.t. $\mathcal{R}(x, w) = 1$;
- Two proof goals:
  - Proof of membership: prove $x \in \mathcal{L}$
  - proof of knowledge: prove "I know $w$ s.t. $\mathcal{R}(x, w) = 1$".

# Soundness Property

**Proof of membership:** if $x \notin \mathcal{L}$, no cheating prover $P^*$ can convince $V$ (except with some small probability).

**Proof of knowledge:** there exist an efficient **Extractor** $E$ s.t.:

- given access to a prover $P$ that convinces $V$ on some input $x$ (with some non-negligible probability)
- can compute some $w$ s.t. $\mathcal{R}(x, w) = 1$ (with some non-negligible probability)

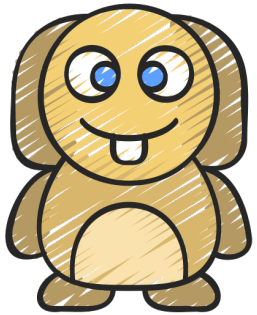


# Soundness Property

**Proof of membership:** if $x \notin \mathcal{L}$, no cheating prover $P^*$ can convince $V$ (except with some small probability).

**Proof of knowledge:** there exist an efficient **Extractor** $E$ s.t.:

- given access to a prover $P$ that convinces $V$ on some input $x$ (with some non-negligible probability)
- can compute some $w$ s.t. $\mathcal{R}(x, w) = 1$ (with some non-negligible probability)

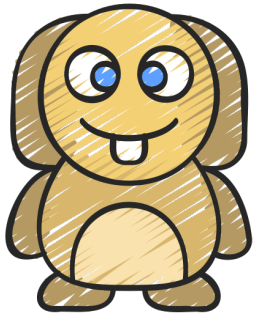


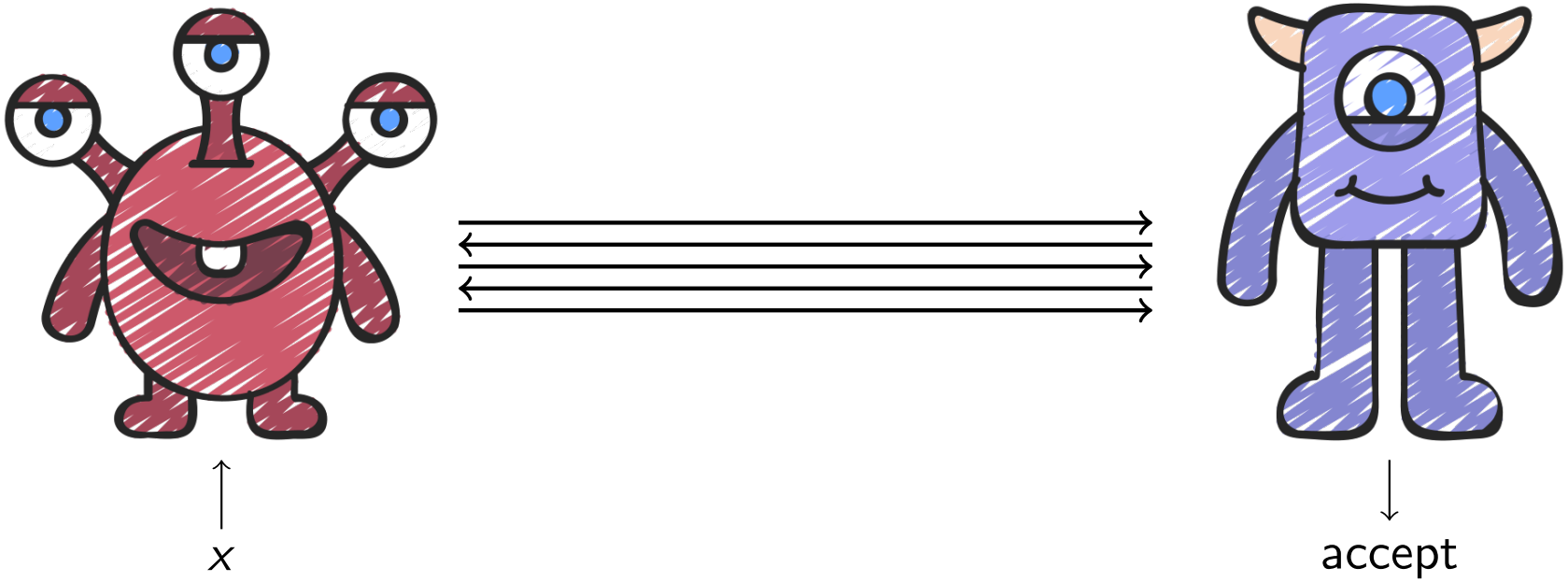
# Soundness Property

**Proof of membership:** if $x \notin \mathcal{L}$, no cheating prover $P^*$ can convince $V$ (except with some small probability).

**Proof of knowledge:** there exist an efficient **Extractor** $E$ s.t.:

- given access to a prover $P$ that convinces $V$ on some input $x$ (with some non-negligible probability)
- can compute some $w$ s.t. $\mathcal{R}(x, w) = 1$ (with some non-negligible probability)



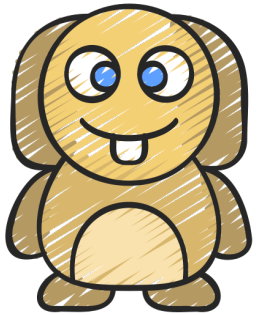


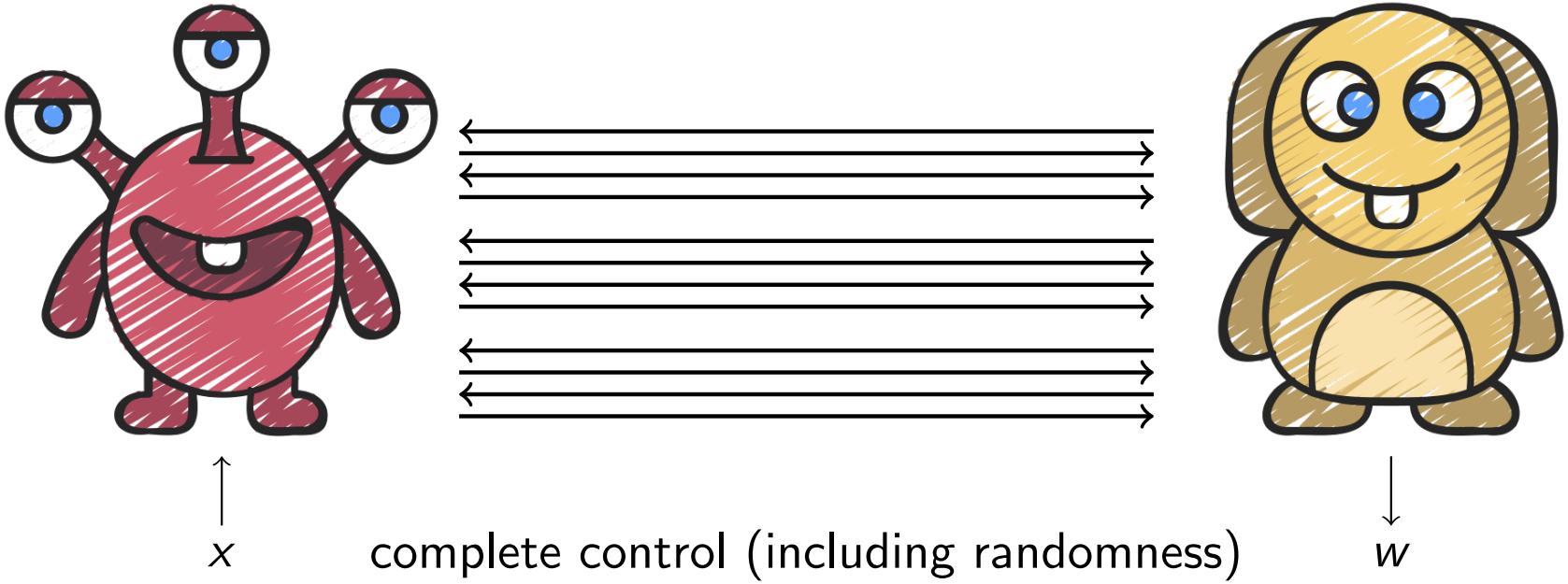
# Soundness Property

**Proof of membership:** if $x \notin \mathcal{L}$, no cheating prover $P^*$ can convince $V$ (except with some small probability).

**Proof of knowledge:** there exist an efficient **Extractor** $E$ s.t.:

- given access to a prover $P$ that convinces $V$ on some input $x$ (with some non-negligible probability)
- can compute some $w$ s.t. $\mathcal{R}(x, w) = 1$ (with some non-negligible probability)



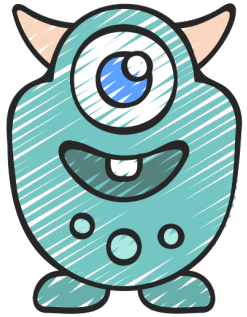


# Zero-Knowledge Property

**Zero-knowledge:** if the statement is true, no cheating verifier learns anything other than this fact.

How? construct a **simulator** (= PPTM)

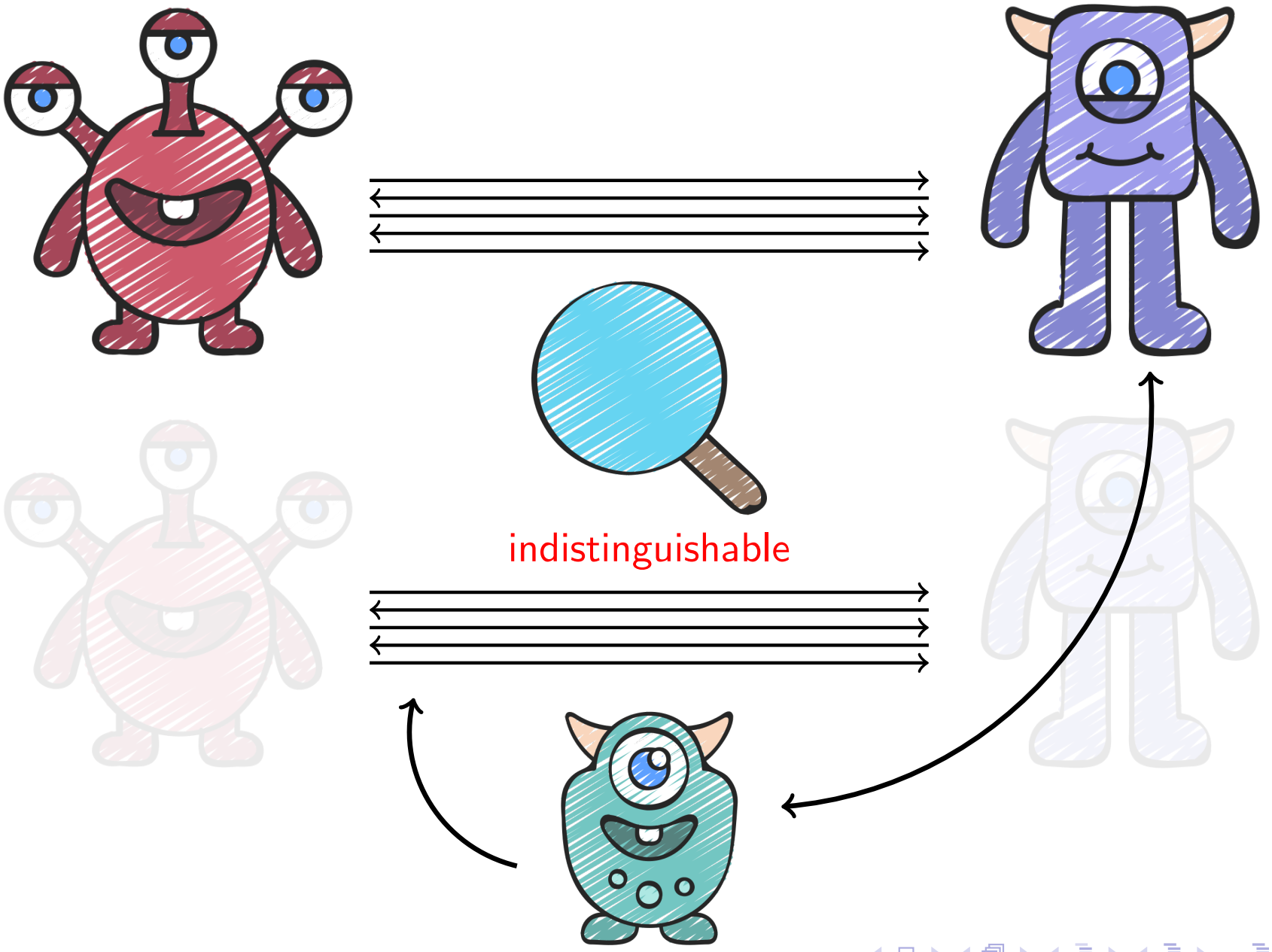
- takes as input what the verifier knows before the protocol starts,
- produces a fake "transcript" of protocol messages that is **indistinguishable** from actual protocol messages



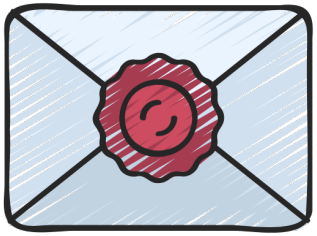
All messages can be simulated from verifier's initial knowledge
$\Rightarrow$ verifier does not learn anything that they did not know before

**Honest-verifier ZK** $\rightsquigarrow$ only considers verifiers that follow the protocol

# Zero-Knowledge Property

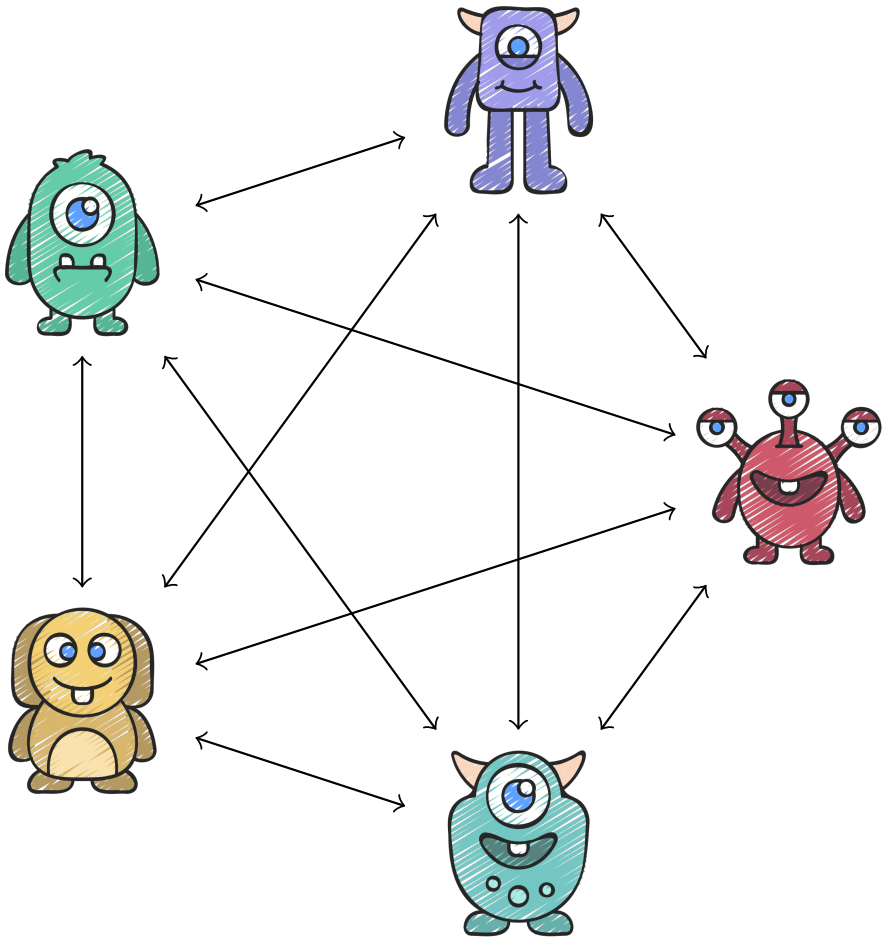


# Commitments



- digital analogue of a sealed envelope
  $\rightsquigarrow$ hide a value that cannot be changed
- (Commit, Open)
  - $\text{Commit}(m; r) \rightsquigarrow (c, s)$
  - $\text{Open}(c, s) \rightsquigarrow m$ or $\perp$
- **Hiding property:** $c$ does not reveal information on $m$
- **Binding property:** impossible to open a commitment to two different values

# Multi-Party Computation



- computation between parties who do not trust each other
- preserve the privacy of each player's inputs
- guarantee the correctness of the computation

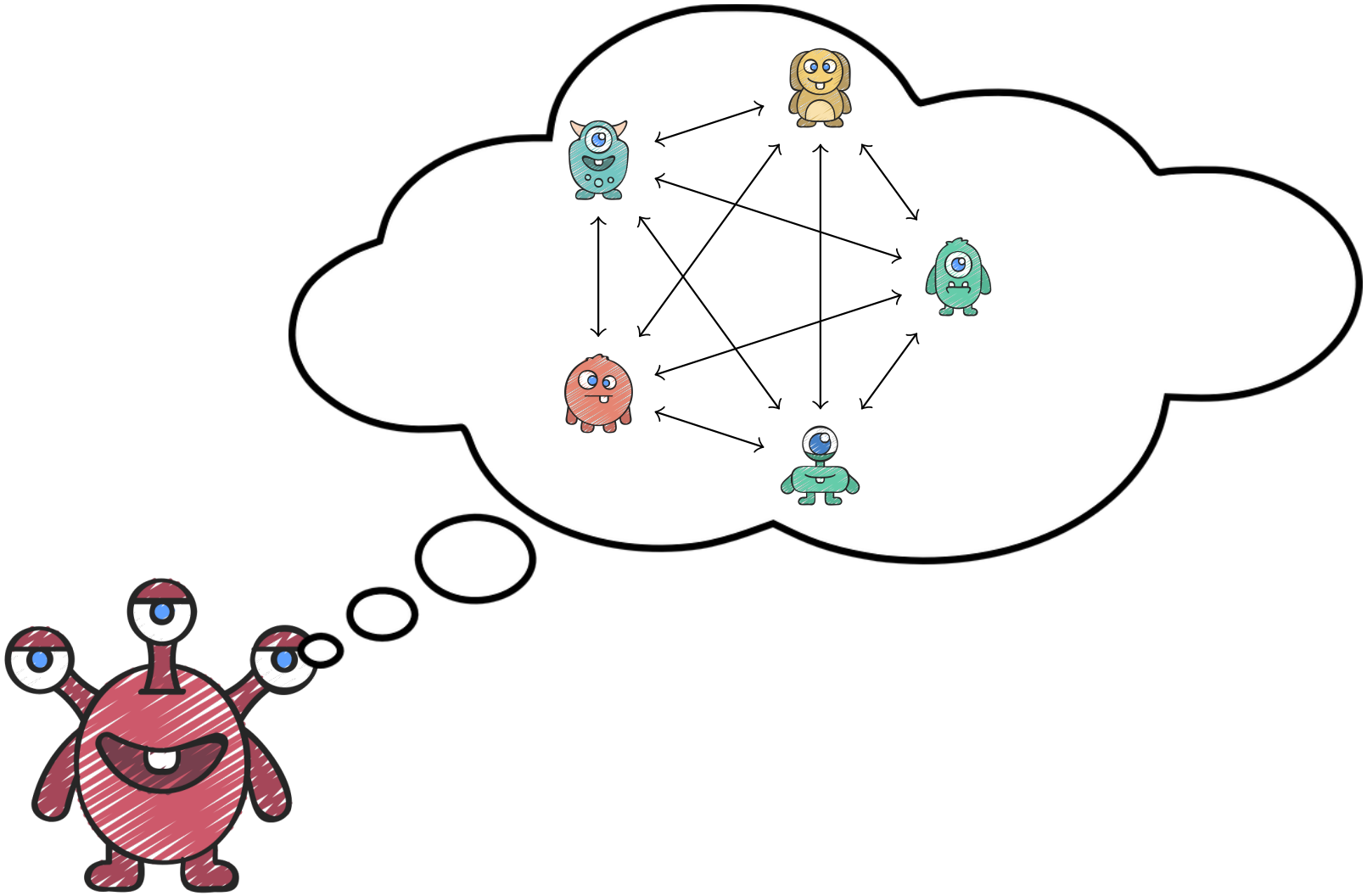
## Semi-Honest Adversaries

For every function $f(x_1, \ldots, x_n)$, there exists a protocol for computing with perfect security in the presence of a semi-honest adversary controlling $t < n/2$ parties

# Outline


1. Picnic Signatures - Overview
2. Zero-Knowledge Proofs and MPC
   - Zero-knowledge (recall)
   - MPC (recall)
3. MPC-in-the-Head
   - ZK from 2-private MPC
   - ZK from 1-private MPC
   - ZK from $n$-private MPC

# MPC-in-the-Head



$\rightsquigarrow$ a technique for zero-knowledge based on MPC

# MPC-in-the-Head

Ishai, Kushilevitz, Ostrovsky, Sahai (2007)
Zero-knowledge from secure multiparty computation. STOC 2007
pp. 21-30

- $\mathcal{L}$ a language in NP (i.e. $x \in \mathcal{L} \iff \exists w, \mathcal{R}(x, w) = 1$)
- Given a public $x$, Alice wants to prove that she knows $w$ s.t.

$$\mathcal{R}(x, w) = 1$$

- Consider an $n$-party computation:

$$g(x; w_1, \ldots, w_n) = \mathcal{R}(x, w_1 \oplus \cdots \oplus w_n)$$

- Alice simulates (in her head) a secure MPC protocol for $g$ with
  - **security in the semi-honest model**
  - **2-privacy**
  - **perfect correctness**

# Views of Parties in MPC

- view of $P_i$ denoted $V_i$ is
  - its input $w_i$
  - its random coins $r_i$
  - all the messages received by $P_i$ (in particular, $g(x, w_1, \ldots, w_n)$)
- Given $V_i$ one can perform the same computation as $P_i$ (using the description of the MPC protocol)
- $V_i$ and $V_j$ are consistent if the outgoing messages $P_i \rightarrow P_j$ are identical to the incoming messages $P_j \leftarrow P_i$ (and *vice versa*)
- Proposition 1: All pairs of views $(V_i, V_j)$ are consistent iff there exists an honest execution of the protocol in which the view of $P_i$ is $V_i$.

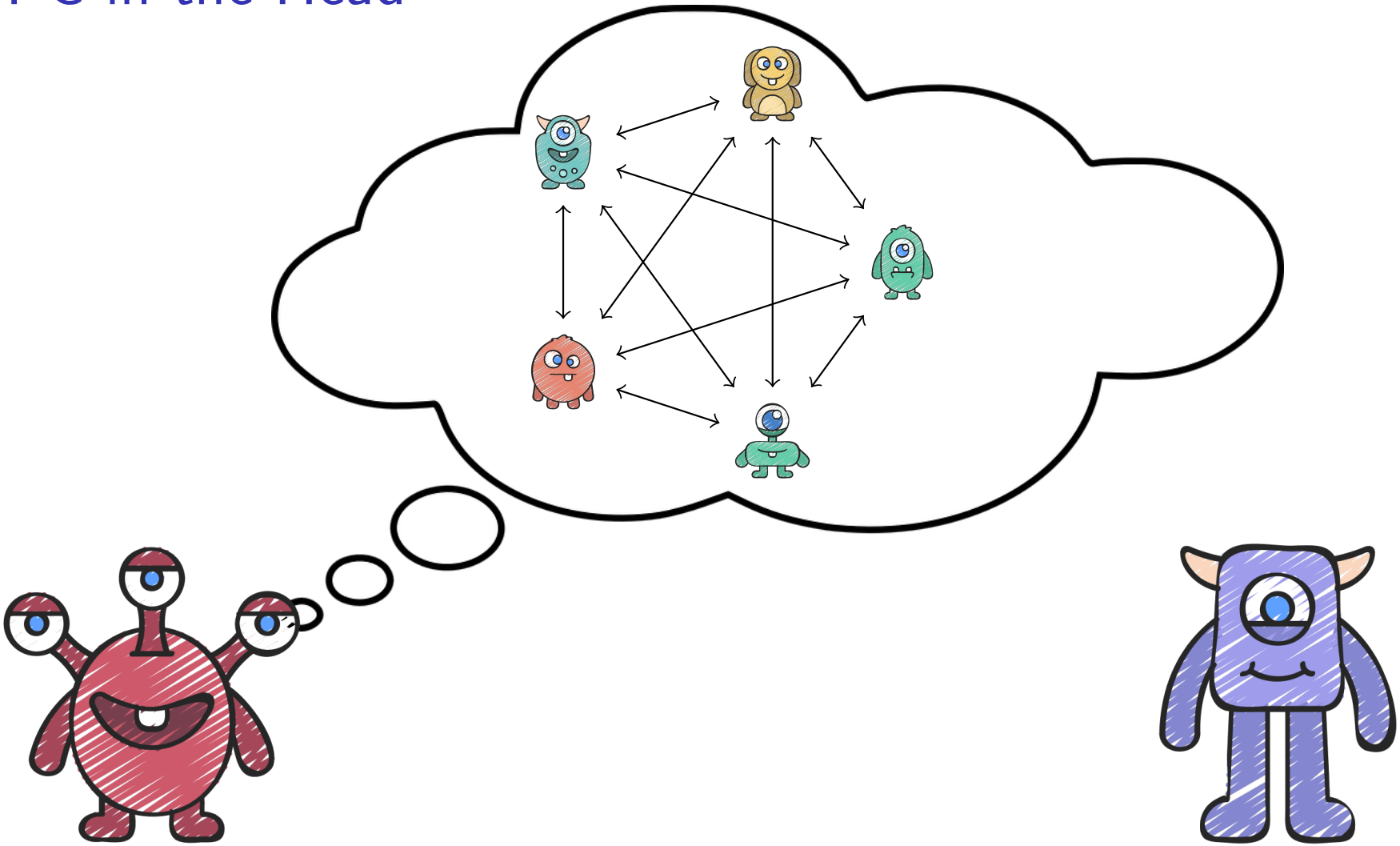
# Views of Parties in MPC

- view of $P_i$ denoted $V_i$ is
  - its input $w_i$
  - its random coins $r_i$
  - all the messages received by $P_i$ (in particular, $g(x, w_1, \ldots, w_n)$)
- Given $V_i$ one can perform the same computation as $P_i$ (using the description of the MPC protocol)
- $V_i$ and $V_j$ are consistent if the outgoing messages $P_i \rightarrow P_j$ are identical to the incoming messages $P_j \leftarrow P_i$ (and *vice versa*)
- Proposition 1: All pairs of views $(V_i, V_j)$ are consistent iff there exists an honest execution of the protocol in which the view of $P_i$ is $V_i$.

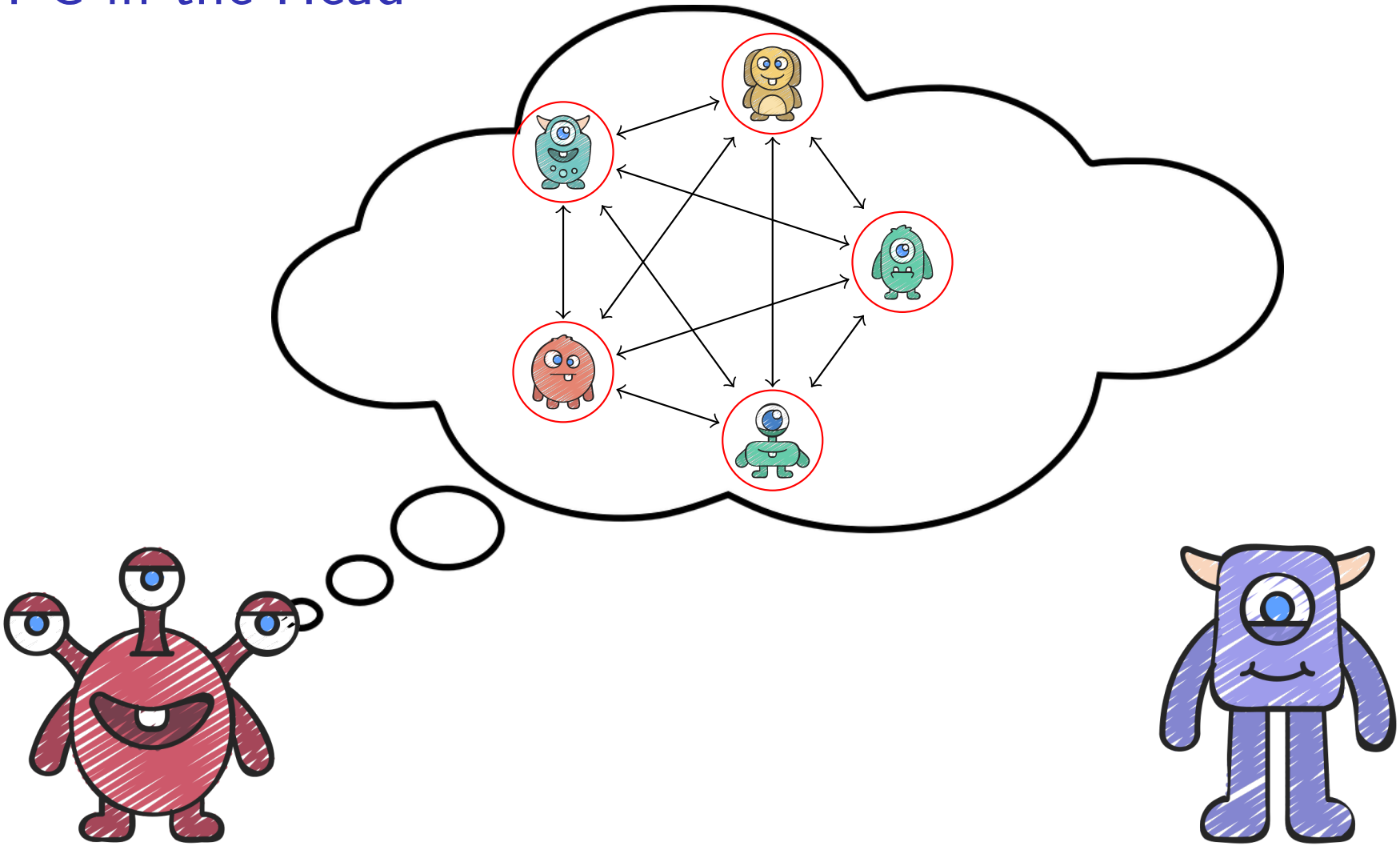
# Views of Parties in MPC

- view of $P_i$ denoted $V_i$ is
  - its input $w_i$
  - its random coins $r_i$
  - all the messages received by $P_i$ (in particular, $g(x, w_1, \ldots, w_n)$)
- Given $V_i$ one can perform the same computation as $P_i$ (using the description of the MPC protocol)
- $V_i$ and $V_j$ are consistent if the outgoing messages $P_i \rightarrow P_j$ are identical to the incoming messages $P_j \leftarrow P_i$ (and *vice versa*)
- Proposition 1: All pairs of views $(V_i, V_j)$ are consistent iff there exists an honest execution of the protocol in which the view of $P_i$ is $V_i$.

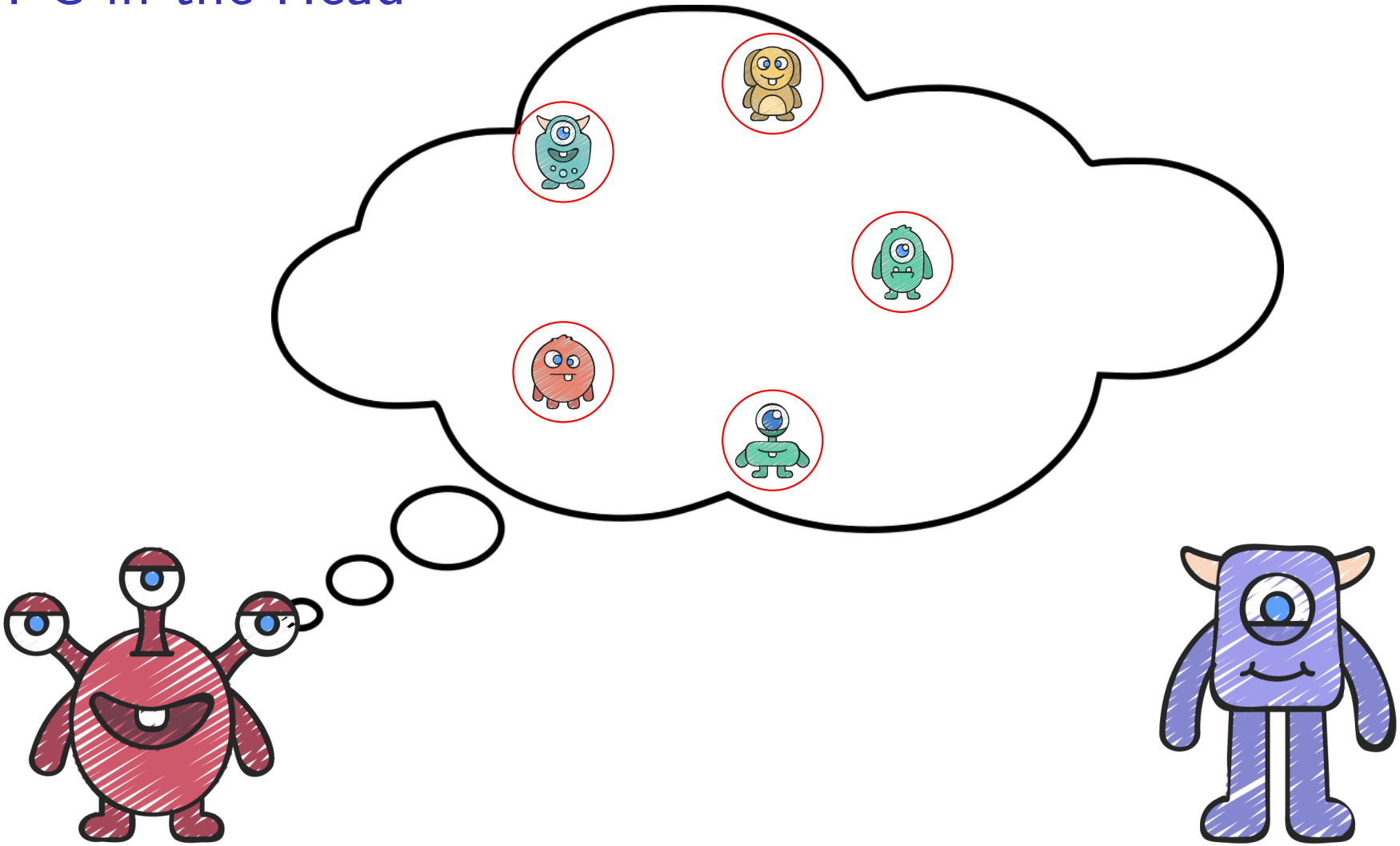
# MPC-in-the-Head



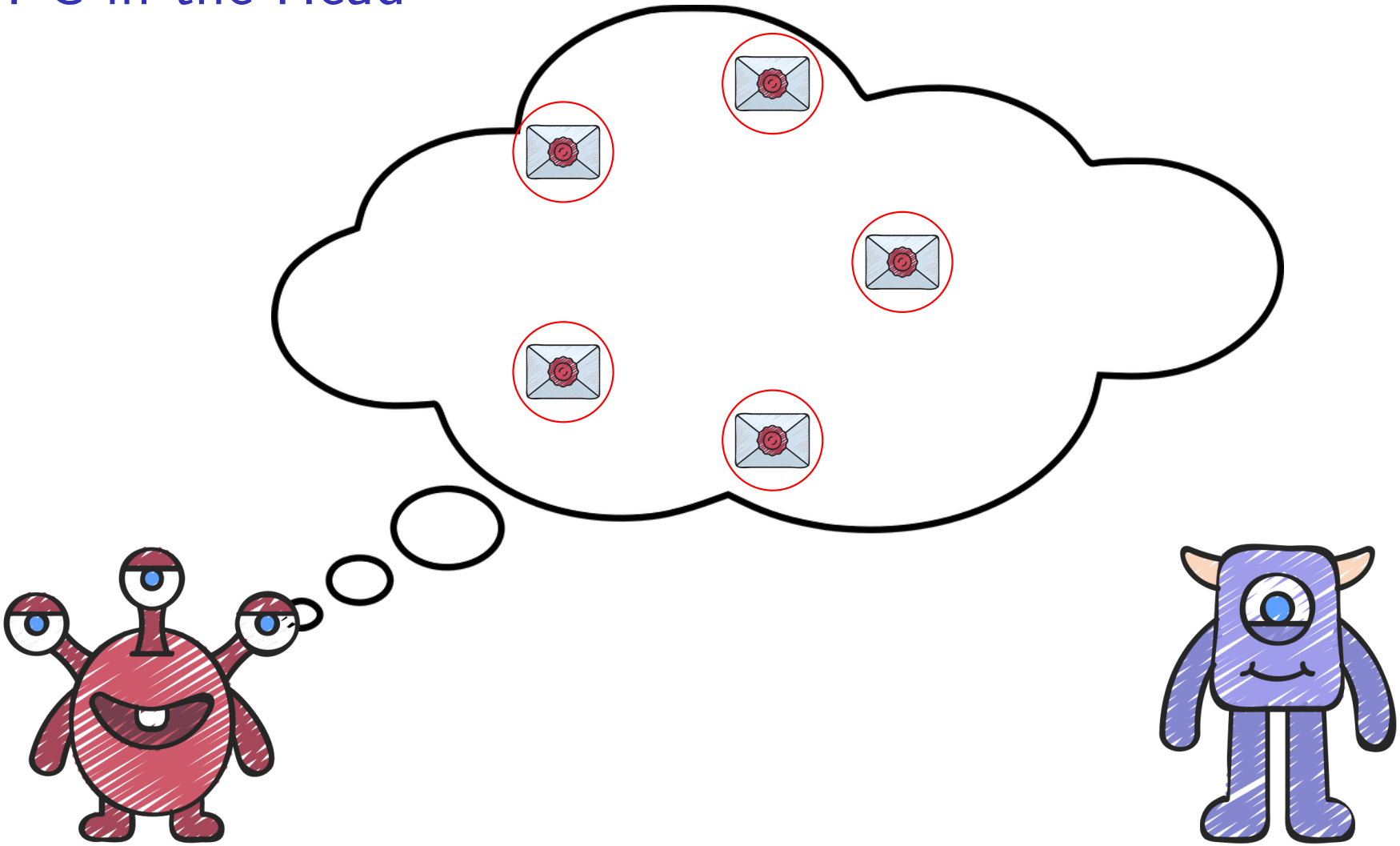
# MPC-in-the-Head



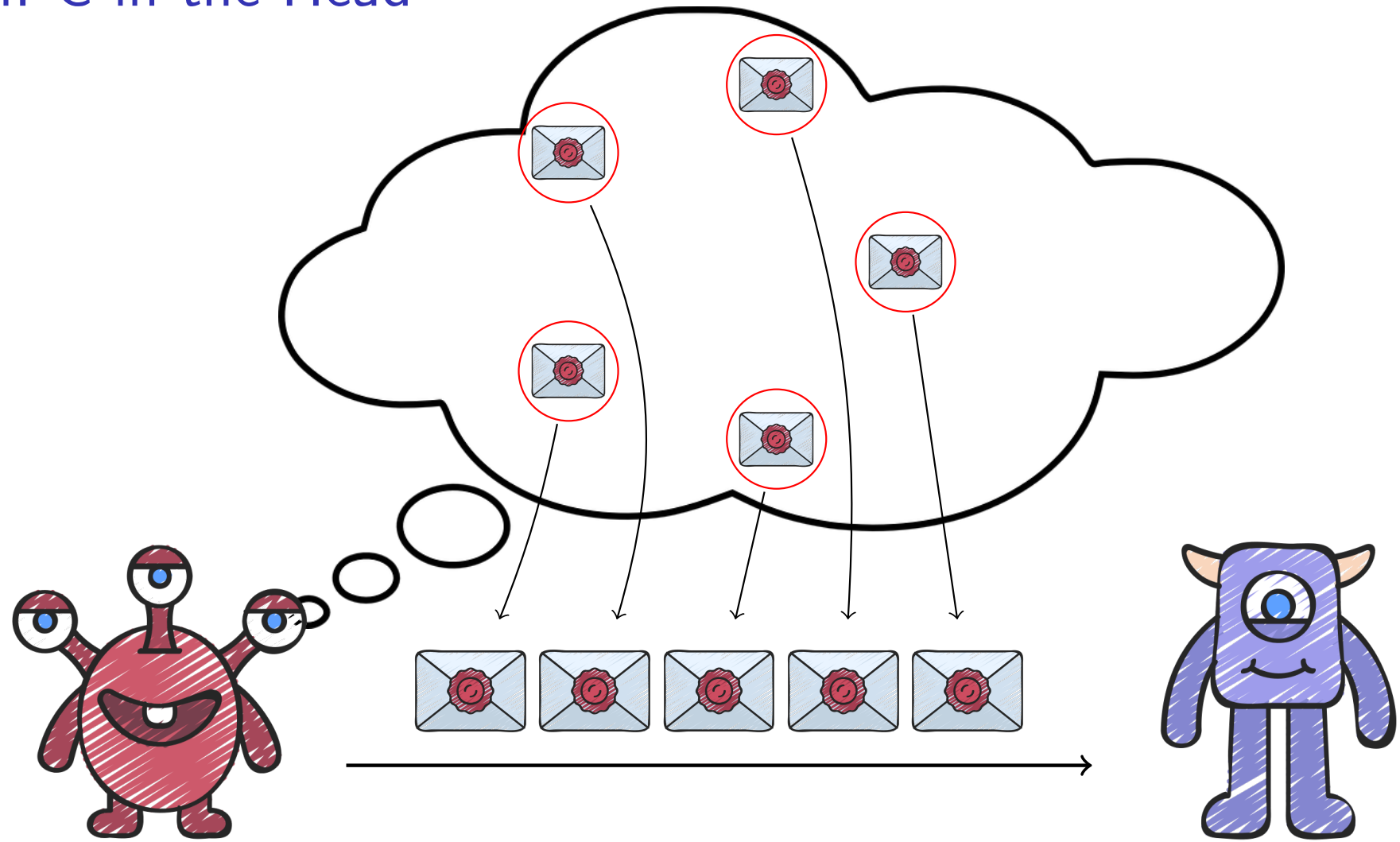
# MPC-in-the-Head



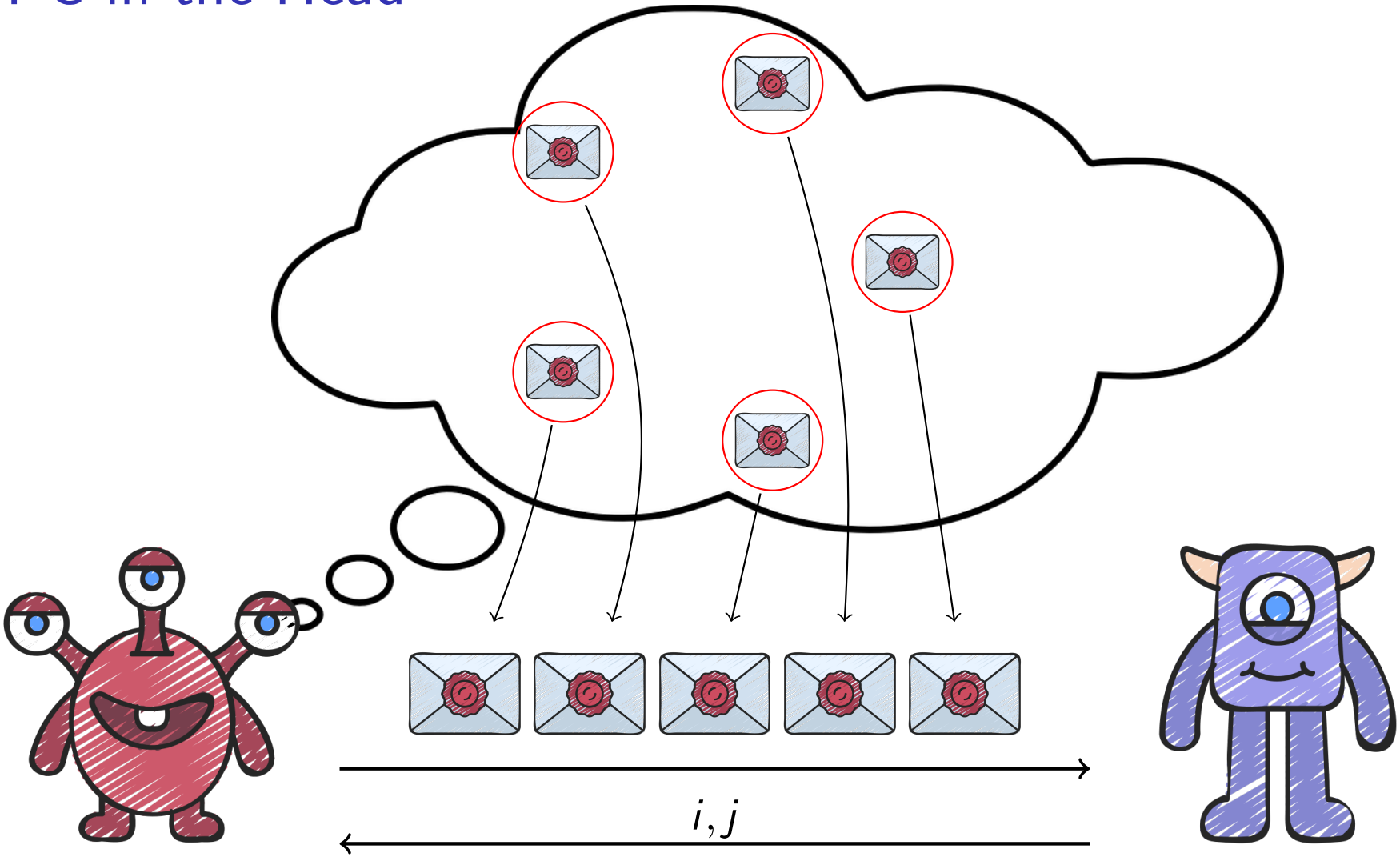
# MPC-in-the-Head



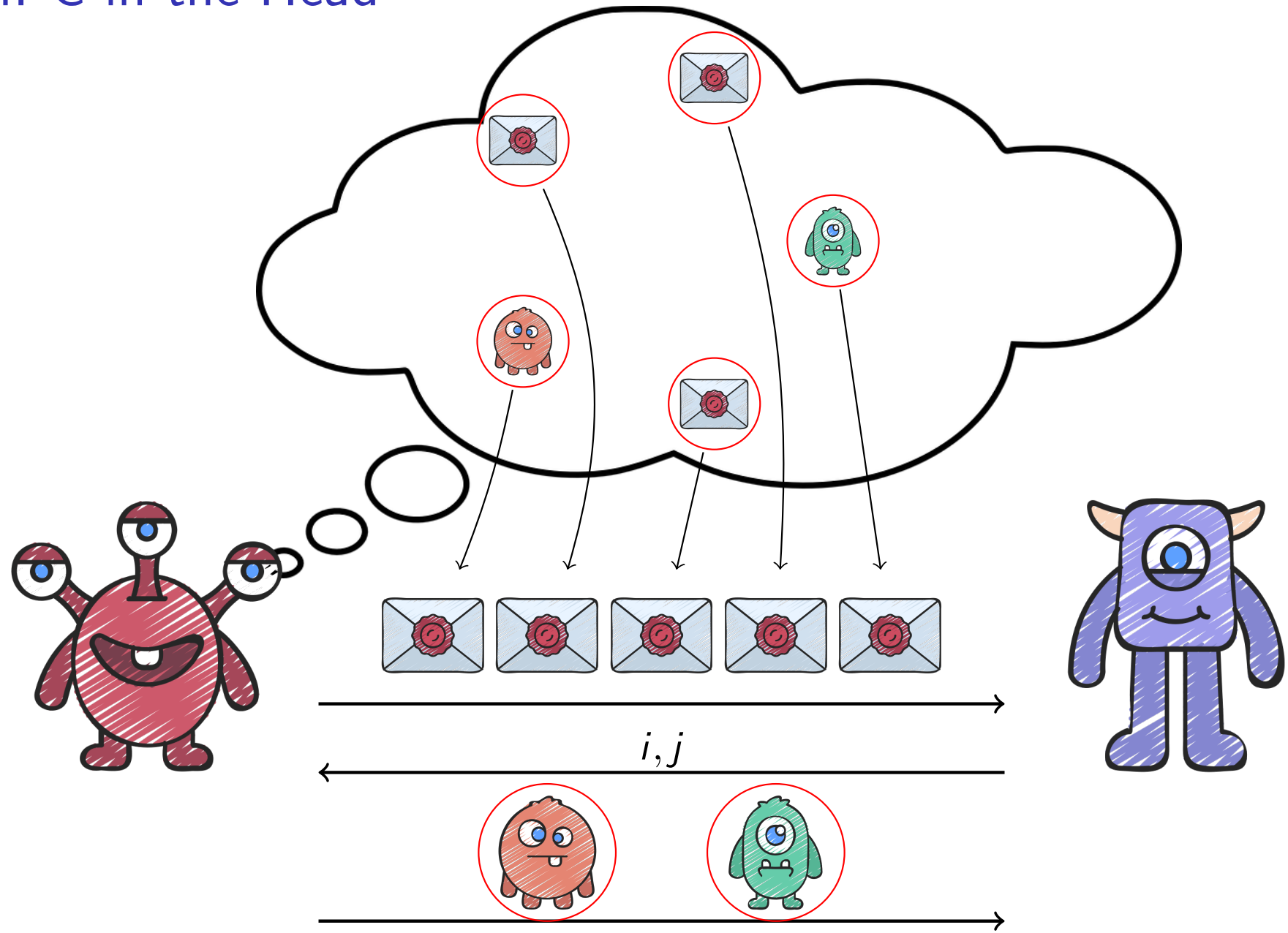
# MPC-in-the-Head



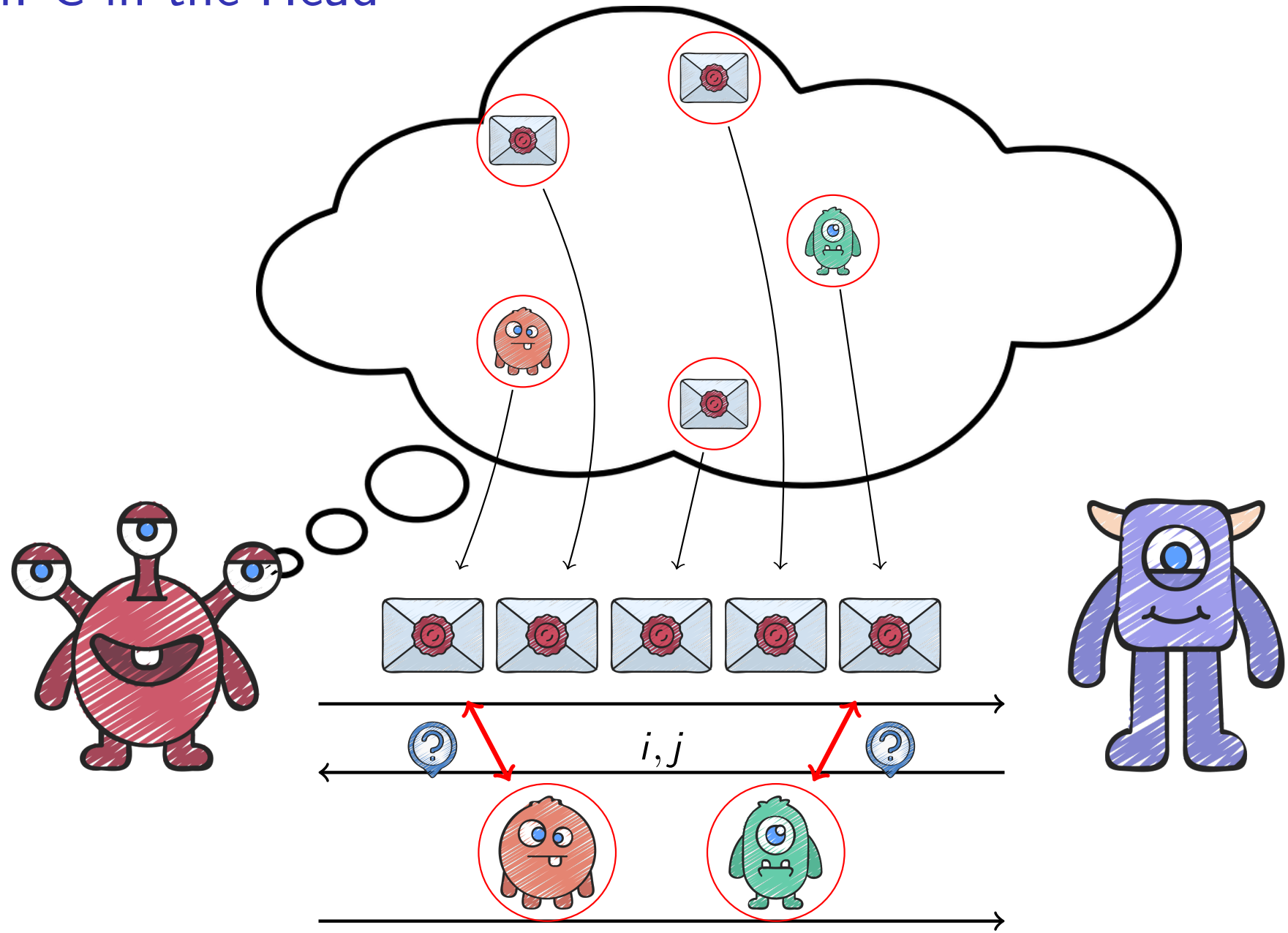
# MPC-in-the-Head



# MPC-in-the-Head



# MPC-in-the-Head



# MPC-in-the-Head

1. Let $\Pi$ be a secure MPC $n$-party protocol for

$$g(x; w_1, \ldots, w_n) = \mathcal{R}(x, w_1 \oplus \cdots \oplus w_n)$$

2. Alice shares her witness $w$ as $w = w_1 \oplus \cdots \oplus w_n$
3. She runs $\Pi$ with $n$ parties $P_1, \ldots, P_n$ using some randomness and $(w_1, \ldots, w_n)$ as private inputs
   $\rightsquigarrow$ views $V_1, \ldots, V_n$
4. Alice computes a **commitment** $C_i$ of $V_i$ $(\forall i)$ and sends $(C_1, \ldots, C_n)$ to Bob
5. Bob chooses randomly $i, j \in \{1, \ldots, n\}$ with $i \neq j$ and sends them to Alice
6. Alice opens $C_i$ and $C_j$ and reveals $V_i$ and $V_j$
7. Bob checks if the commitments were correct, if the views are consistent and if the ouput $g(x, w_1, \ldots, w_n) = 1$.

# MPC-in-the-Head – Security

1. **Completeness:** by inspection

2. **Soundness:** by Proposition 1, if all pairs of views are consistent and $\Pi$ outputs 1 then

$$g(x, w_1, \ldots, w_n) = 1 \rightsquigarrow \mathcal{R}(x, w) = 1 \rightsquigarrow x \in \mathcal{L}$$

If $x \notin \mathcal{L}$, either $g(x, w_1, \ldots, w_n) = 0$ or (at least) one pair of views is inconsistent.This is detected with probability at least

$$\binom{n}{2}^{-1} = \frac{2}{n(n-1)}$$

3. **Zero-knowledge:** by the hiding property of the commitment scheme and the 2-privacy of $\Pi$

# MPC-in-the-Head using BGW

- the complexity **increases** with $n$
  the cheating probability **increases** with $n$
  $\rightsquigarrow$ pick the minimal $n$!
- for 2-privacy with BGW, we need at least $n \geq 5$ players (honest majority)
- with $n = 5$,
  - Alice has to commit 5 views of the protocol (and reveals 2)
  - If she cheats, Bob detects it with probability $\geq 1/10$
- with $n = 5$, a cheating Alice is not detected
  - in one run with probability $\leq 9/10$
  - in $k$ independent runs with probability $\leq (9/10)^k$
    $\rightsquigarrow$ with $k \geq 842$, Alice is not detected with probability $\leq 2^{-128}$

# MPC-in-the-Head using BGW

- the complexity **increases** with $n$
  the cheating probability **increases** with $n$
  $\rightsquigarrow$ pick the minimal $n$!
- for 2-privacy with BGW, we need at least $n \geq 5$ players (honest majority)
- with $n = 5$,
  - Alice has to commit 5 views of the protocol (and reveals 2)
  - If she cheats, Bob detects it with probability $\geq 1/10$
- with $n = 5$, a cheating Alice is not detected
  - in one run with probability $\leq 9/10$
  - in $k$ independent runs with probability $\leq (9/10)^k$
    $\rightsquigarrow$ with $k \geq 842$, Alice is not detected with probability $\leq 2^{-128}$

# MPC-in-the-Head using BGW

- the complexity **increases** with $n$
  the cheating probability **increases** with $n$
  $\rightsquigarrow$ pick the minimal $n$!
- for 2-privacy with BGW, we need at least $n \geq 5$ players (honest majority)
- with $n = 5$,
  - Alice has to commit 5 views of the protocol (and reveals 2)
  - If she cheats, Bob detects it with probability $\geq 1/10$
- with $n = 5$, a cheating Alice is not detected
  - in one run with probability $\leq 9/10$
  - in $k$ independent runs with probability $\leq (9/10)^k$
    $\rightsquigarrow$ with $k \geq 842$, Alice is not detected with probability $\leq 2^{-128}$

# MPC-in-the-Head using BGW

- the complexity **increases** with $n$
  the cheating probability **increases** with $n$
  $\rightsquigarrow$ pick the minimal $n$!
- for 2-privacy with BGW, we need at least $n \geq 5$ players (honest majority)
- with $n = 5$,
  - Alice has to commit 5 views of the protocol (and reveals 2)
  - If she cheats, Bob detects it with probability $\geq 1/10$
- with $n = 5$, a cheating Alice is not detected
  - in one run with probability $\leq 9/10$
  - in $k$ independent runs with probability $\leq (9/10)^k$
    $\rightsquigarrow$ with $k \geq 842$, Alice is not detected with probability $\leq 2^{-128}$

# MPC-in-the-Head from 1-private MPC

- **Idea:** use a simpler MPC protocol with 1-privacy to improve efficiency
- one can reveal only one view $V_i$ of the protocol to achieve ZK
  $\rightsquigarrow$ this proves nothing (no soundess)
- We have a proposition similar to the previous one:

  Proposition 2: All views $V_i$ are consistent with the $(n-1)$ channels incident to $P_i$ iff there exists an honest execution of the protocol in which the view of $P_i$ is $V_i$.

  $\rightsquigarrow$ In addition, we reveal all messages sent/received by $P_i$ over the $n-1$ channels with the other parties

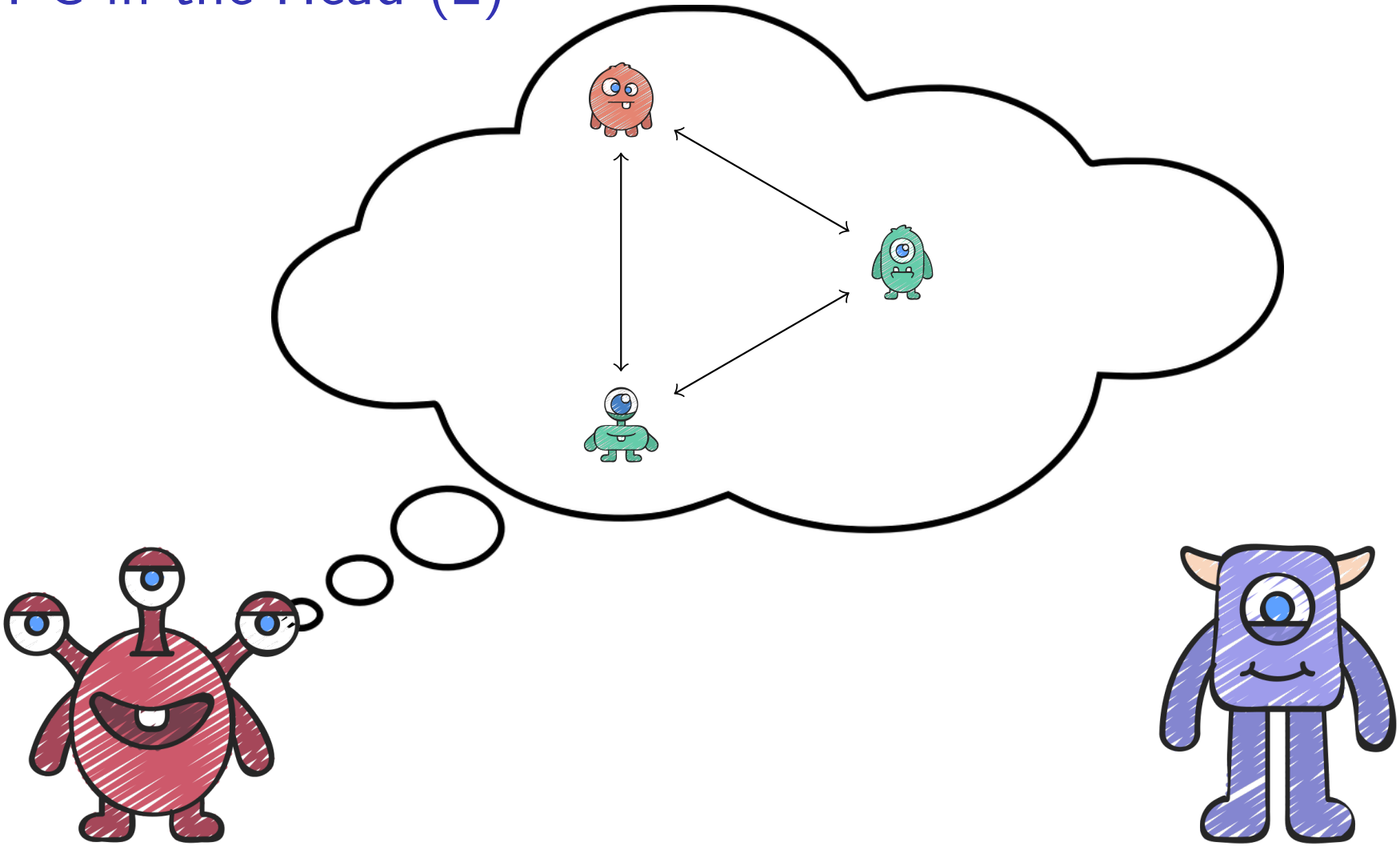
# MPC-in-the-Head from 1-private MPC

- **Idea:** use a simpler MPC protocol with 1-privacy to improve efficiency
- one can reveal only one view $V_i$ of the protocol to achieve ZK
  $\rightsquigarrow$ this proves nothing (no soundess)
- We have a proposition similar to the previous one:

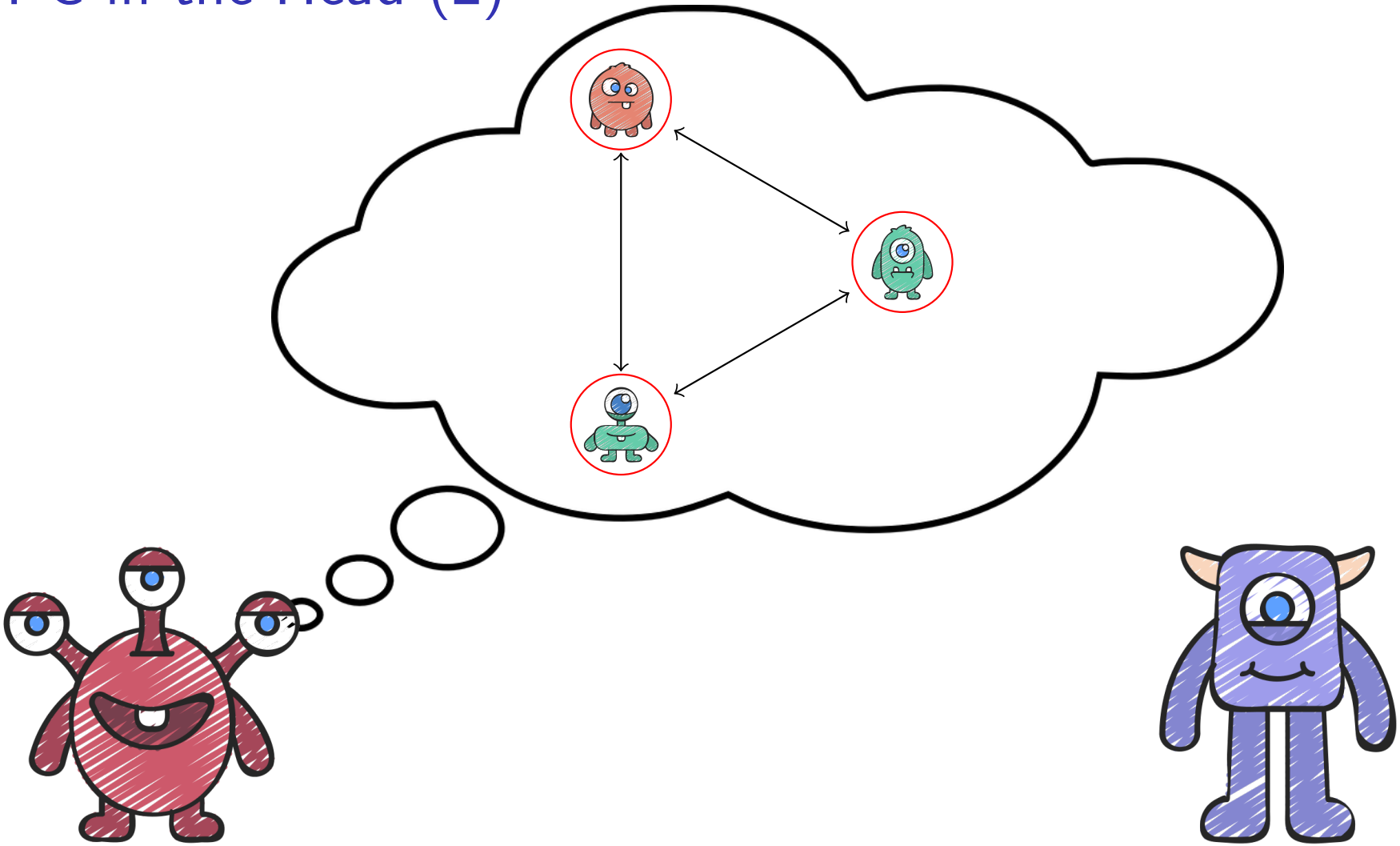
  Proposition 2: All views $V_i$ are consistent with the $(n-1)$ channels incident to $P_i$ iff there exists an honest execution of the protocol in which the view of $P_i$ is $V_i$.

  $\rightsquigarrow$ In addition, we reveal all messages sent/received by $P_i$ over the $n-1$ channels with the other parties

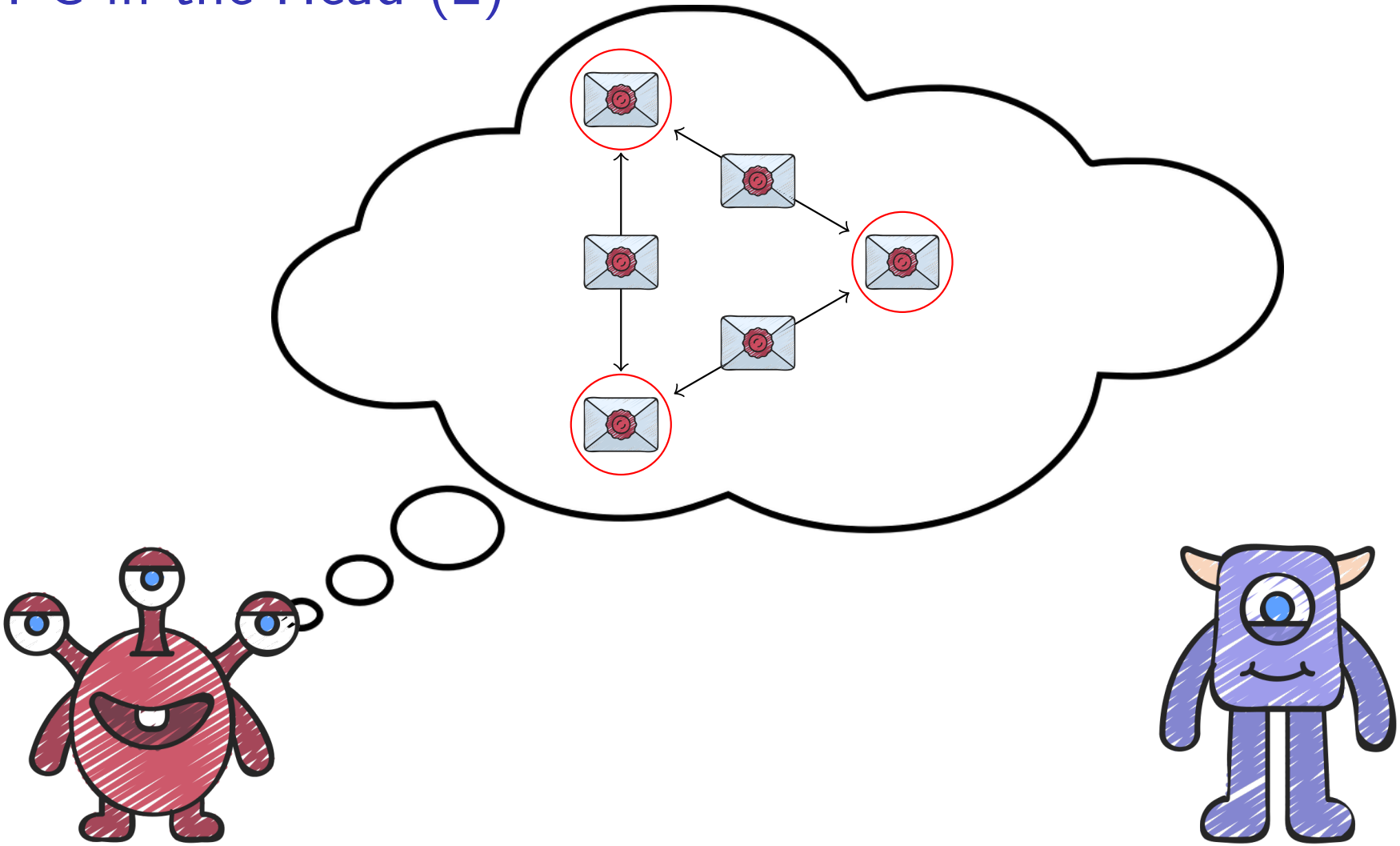
# MPC-in-the-Head (2)



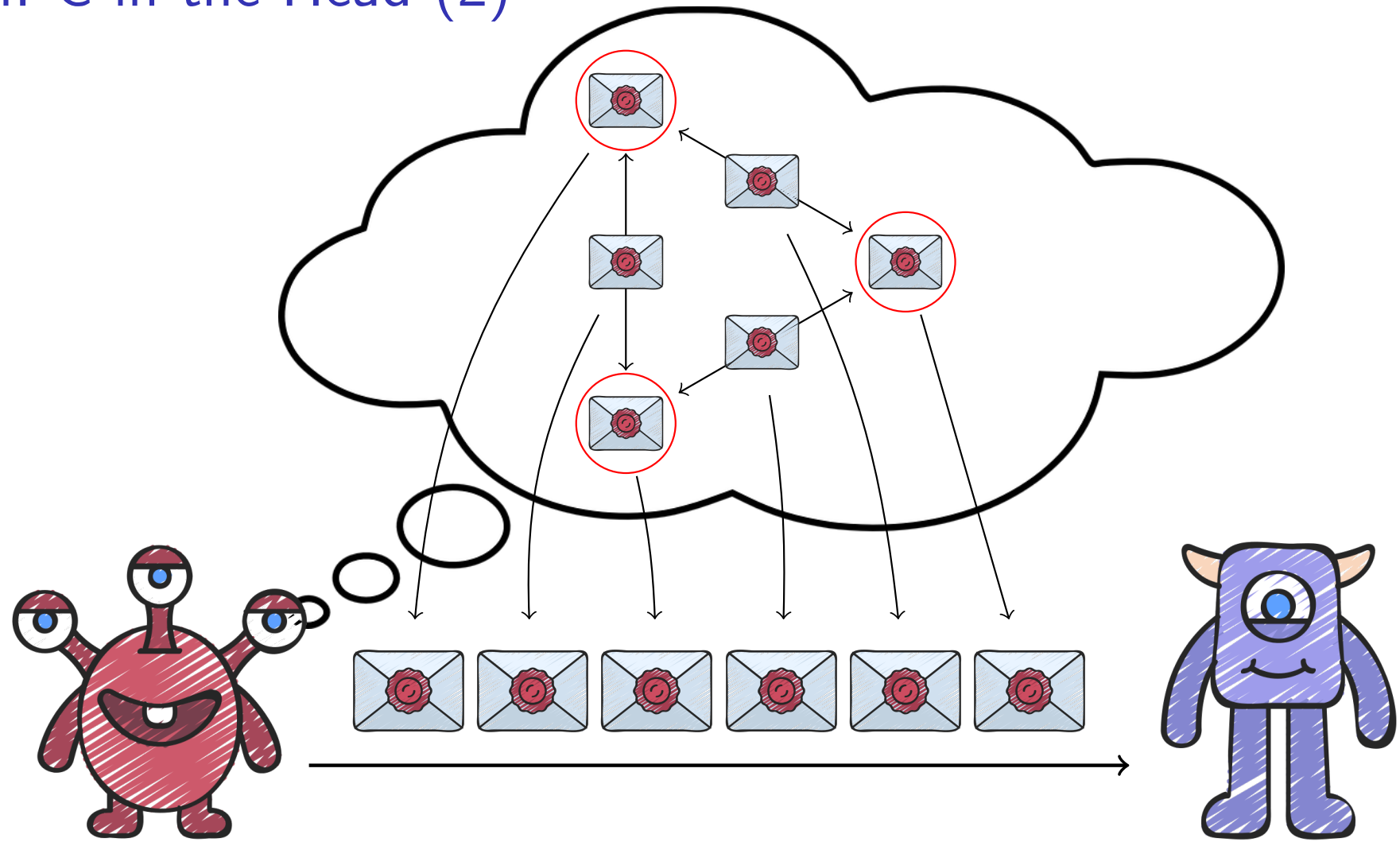
# MPC-in-the-Head (2)



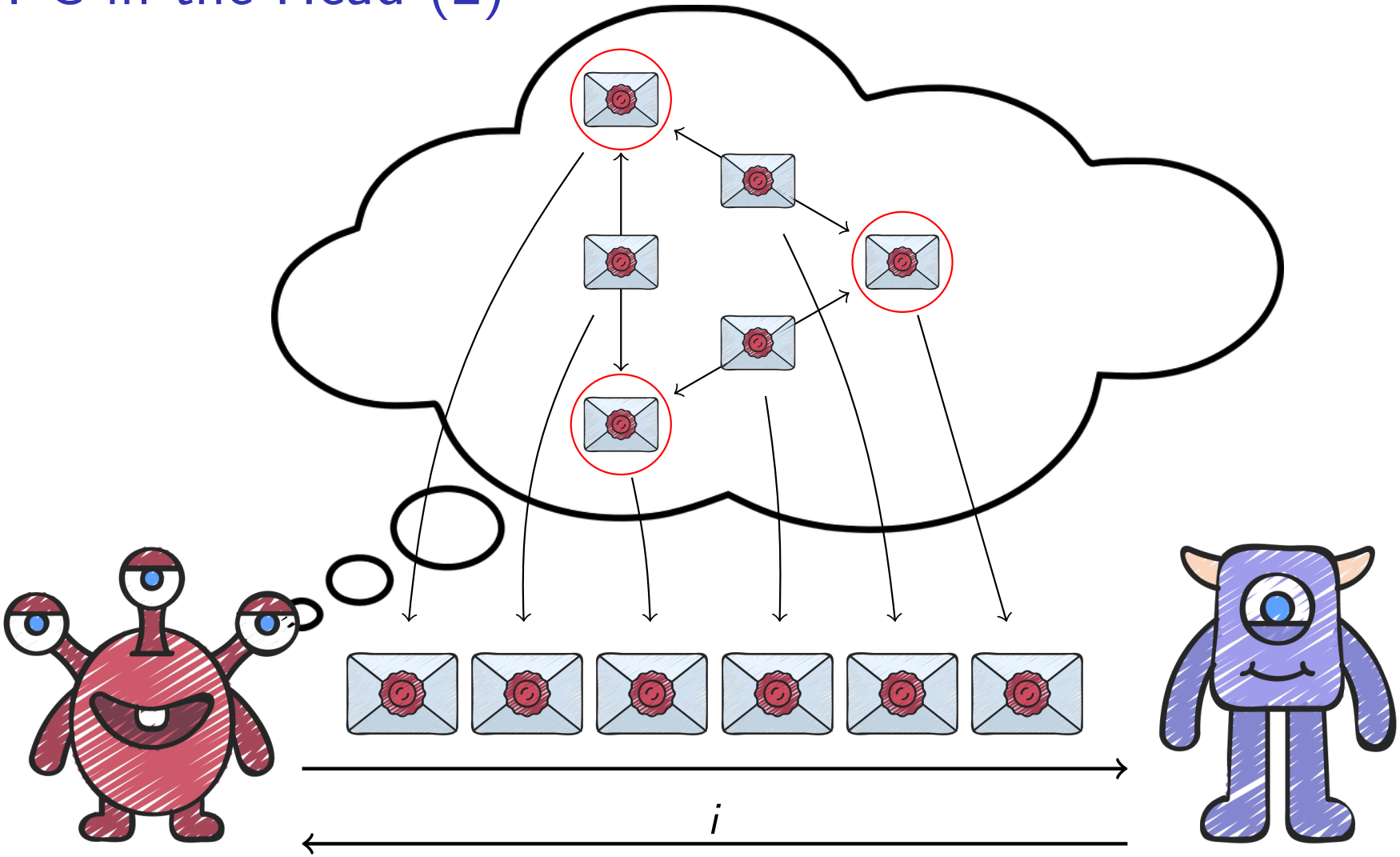
# MPC-in-the-Head (2)



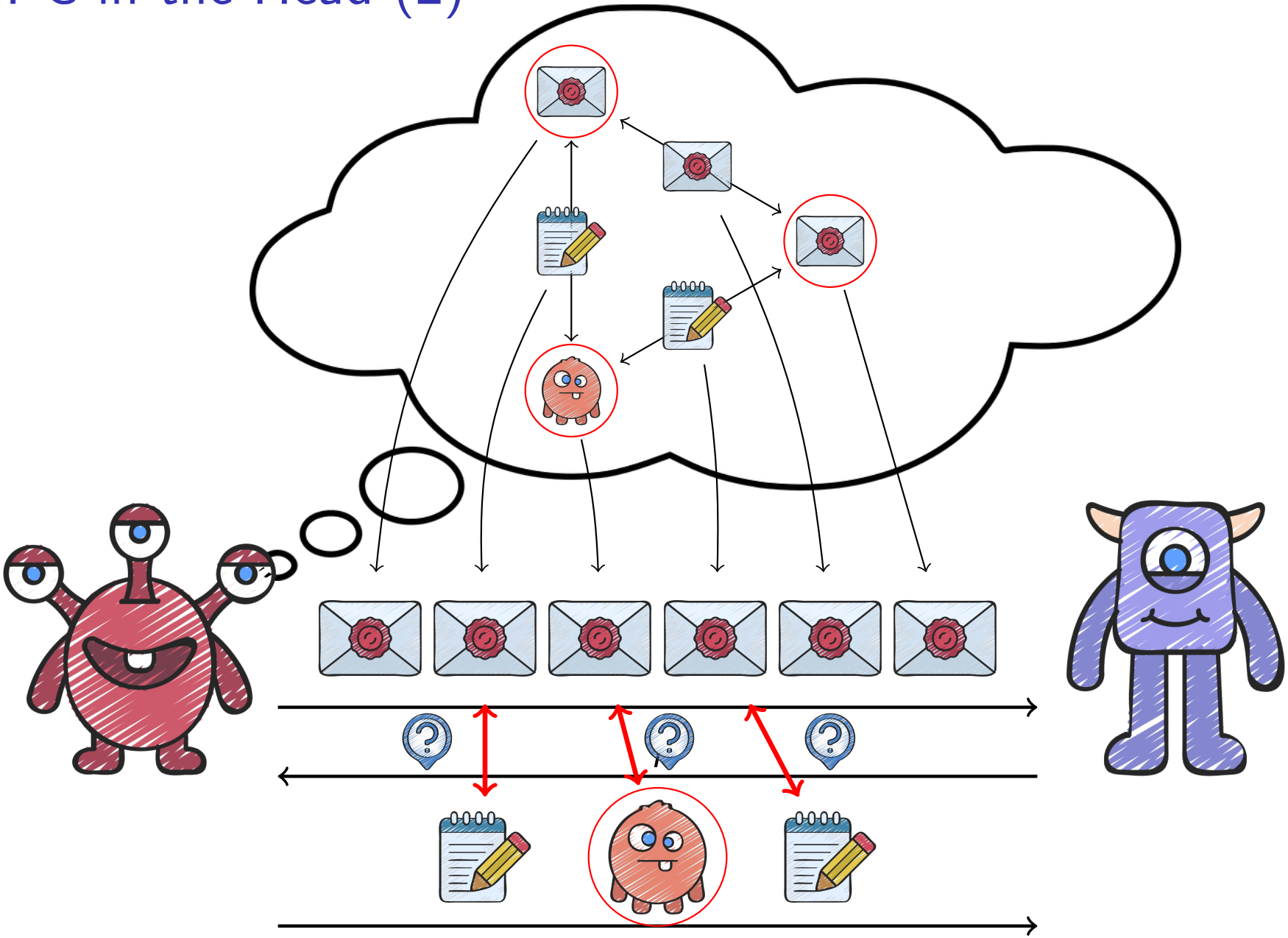
# MPC-in-the-Head (2)



# MPC-in-the-Head (2)



# MPC-in-the-Head (2)



# MPC-in-the-Head from 1-private MPC

1. Let $\Pi$ be a secure MPC $n$-party protocol for

$$g(x; w_1, \ldots, w_n) = \mathcal{R}(x, w_1 \oplus \cdots \oplus w_n)$$

2. Alice shares her witness $w$ as $w = w_1 \oplus \cdots \oplus w_n$
3. She runs $\Pi$ with $n$ parties $P_1, \ldots, P_n$ using some randomness and $(w_1, \ldots, w_n)$ as private inputs
   $\rightsquigarrow$ views $V_1, \ldots, V_n$ / channels $M_{\{1,2\}}, \ldots, M_{\{n-1,n\}}$
4. She computes **commitments** $C_i$ of $V_i$ and $C_{i,j}$ of $M_{i,j}$ $(\forall i, j)$ and sends $(C_1, \ldots, C_n, C_{\{1,2\}}, C_{\{1,3\}}, \ldots, C_{\{n-1,n\}})$ to Bob
5. Bob chooses randomly $i \in \{1, \ldots, n\}$ sends it to Alice
6. Alice opens $C_i$ and $C_{\{i,j\}}$ $(\forall j)$ and reveals $V_i$ and $M_{\{i,j\}}$
7. Bob checks if the commitments were correct, if the views are consistent with the channels and if $g(x, w_1, \ldots, w_n) = 1$.

# MPC-in-the-Head from 1-private MPC – Security

1. **Completeness:** by inspection

2. **Soundness:** by Proposition 2, if all views are consistent with the incident channels and $\Pi$ outputs 1 then

$$g(x, w_1, \ldots, w_n) = 1 \rightsquigarrow \mathcal{R}(x, w) = 1 \rightsquigarrow x \in \mathcal{L}$$

If $x \notin \mathcal{L}$, either $g(x, w_1, \ldots, w_n) = 0$ or (at least) one view is inconsistent with the channels.This is detected with probability at least

$$\frac{1}{n}$$

3. **Zero-knowledge:** by the hiding property of the commitment scheme and the 1-privacy of $\Pi$

# MPC-in-the-Head using BGW

- the complexity **increases** with $n$
  the cheating probability **increases** with $n$
  $\rightsquigarrow$ pick the minimal $n$!
- for 1-privacy with BGW, we need at least $n \geq 3$ players (honest majority)
- with $n = 3$,
  - Alice has to commit 3 views and 3 channels (and reveals 3)
  - If she cheats, Bob detects it with probability $\geq 1/3$
- with $n = 3$, a cheating Alice is not detected
  - in one run with probability $\leq 2/3$
  - in $k$ independent runs with probability $\leq (2/3)^k$
    $\rightsquigarrow$ with $k \geq 219$, Alice is not detected with probability $\leq 2^{-128}$

# MPC-in-the-Head using BGW

- the complexity **increases** with $n$
  the cheating probability **increases** with $n$
  $\rightsquigarrow$ pick the minimal $n$!
- for 1-privacy with BGW, we need at least $n \geq 3$ players (honest majority)
- with $n = 3$,
  - Alice has to commit 3 views and 3 channels (and reveals 3)
  - If she cheats, Bob detects it with probability $\geq 1/3$
- with $n = 3$, a cheating Alice is not detected
  - in one run with probability $\leq 2/3$
  - in $k$ independent runs with probability $\leq (2/3)^k$
    $\rightsquigarrow$ with $k \geq 219$, Alice is not detected with probability $\leq 2^{-128}$

# MPC-in-the-Head using BGW

- the complexity **increases** with $n$
  the cheating probability **increases** with $n$
  $\rightsquigarrow$ pick the minimal $n$!
- for 1-privacy with BGW, we need at least $n \geq 3$ players (honest majority)
- with $n = 3$,
  - Alice has to commit 3 views and 3 channels (and reveals 3)
  - If she cheats, Bob detects it with probability $\geq 1/3$
- with $n = 3$, a cheating Alice is not detected
  - in one run with probability $\leq 2/3$
  - in $k$ independent runs with probability $\leq (2/3)^k$
    $\rightsquigarrow$ with $k \geq 219$, Alice is not detected with probability $\leq 2^{-128}$

# MPC-in-the-Head using BGW

- the complexity **increases** with $n$
  the cheating probability **increases** with $n$
  $\rightsquigarrow$ pick the minimal $n$!
- for 1-privacy with BGW, we need at least $n \geq 3$ players (honest majority)
- with $n = 3$,
  - Alice has to commit 3 views and 3 channels (and reveals 3)
  - If she cheats, Bob detects it with probability $\geq 1/3$
- with $n = 3$, a cheating Alice is not detected
  - in one run with probability $\leq 2/3$
  - in $k$ independent runs with probability $\leq (2/3)^k$
    $\rightsquigarrow$ with $k \geq 219$, Alice is not detected with probability $\leq 2^{-128}$

# $n$-out-of-$n$ Secret Sharing in MPC

- Is it possible to reveal $n-1$ shares in the MPC and remain secure?
  $\rightsquigarrow$ would lead to better soundness!
- impossible classically for general functions (for IT security)
- **but**, possible for "linear" functions, e.g. for $a, b \in \mathbb{Z}$:

$$\begin{aligned} a \cdot x \boxplus b \cdot y &= a \cdot (x_1 \boxplus \cdots \boxplus x_n) \boxplus b \cdot (y_1 \boxplus \cdots \boxplus y_n) \\ &= (a \cdot x_1 \boxplus b \cdot y_1) \boxplus \cdots \boxplus (a \cdot x_n \boxplus b \cdot y_n) \end{aligned}$$

where $a \cdot x = \underbrace{x \boxplus \cdots \boxplus x}_{a \text{ times}}$ (for $a \geq 0$)

# $n$-out-of-$n$ Secret Sharing in MPC

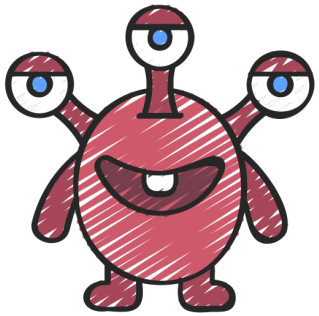
- Is it possible to reveal $n-1$ shares in the MPC and remain secure?
  $\rightsquigarrow$ would lead to better soundness!
- impossible classically for general functions (for IT security)
- **but**, possible for "linear" functions, e.g. for $a, b \in \mathbb{Z}$:

$$\begin{aligned} a \cdot x \boxplus b \cdot y &= a \cdot (x_1 \boxplus \cdots \boxplus x_n) \boxplus b \cdot (y_1 \boxplus \cdots \boxplus y_n) \\ &= (a \cdot x_1 \boxplus b \cdot y_1) \boxplus \cdots \boxplus (a \cdot x_n \boxplus b \cdot y_n) \end{aligned}$$

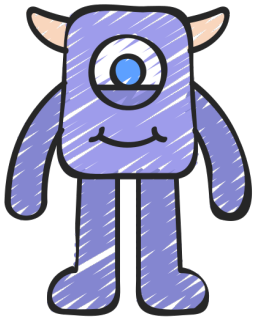
where $a \cdot x = \underbrace{x \boxplus \cdots \boxplus x}_{a \text{ times}}$ (for $a \geq 0$)

# RSA-in-the-head (Maire-V. 2023)

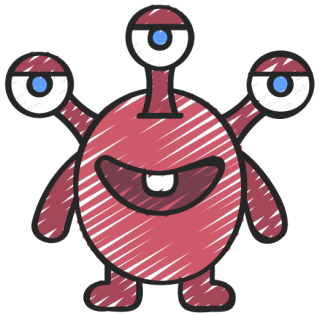
$$\boxed{(N, e) \quad y = x^e \bmod N}$$



$x$ $\longrightarrow$ $y$

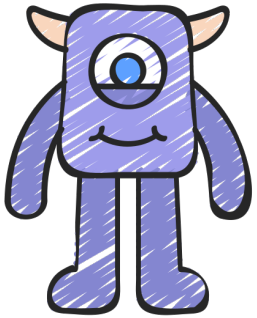


# RSA-in-the-head (Maire-V. 2023)



$$\boxed{(N, e) \quad y = x^e \bmod N}$$

$x$ $\qquad\qquad\qquad\qquad$ $y$

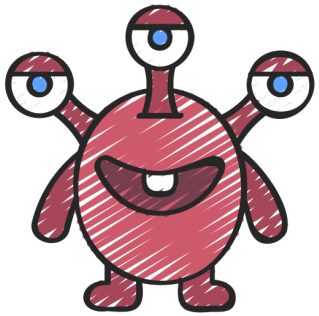


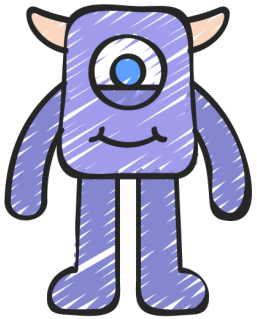
$[\![x]\!] \xleftarrow{\boxed{\cdot\cdot}\boxed{\cdot\cdot}} [(\mathbb{Z}/N\mathbb{Z})^*]^n$

$(y_i \leftarrow x_i^e \bmod N)_i$

# RSA-in-the-head (Maire-V. 2023)

$\boxed{(N, e) \quad y = x^e \bmod N}$





$x$ $\qquad\qquad\qquad\qquad$ $y$

$[\![x]\!] \xleftarrow{\boxed{\cdot\cdot}\boxed{\cdot\cdot}} [(\mathbb{Z}/N\mathbb{Z})^*]^n$

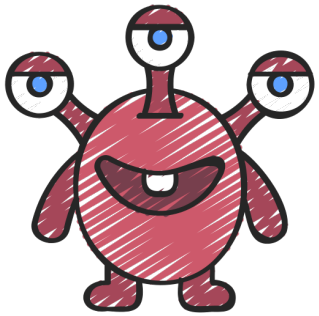
$(y_i \leftarrow x_i^e \bmod N)_i$

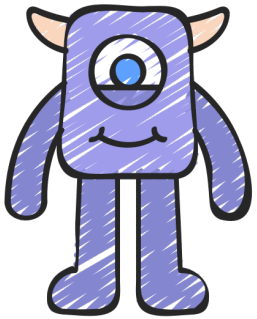
$\xrightarrow{\vec{y}}$

$i^\star \xleftarrow{\boxed{\cdot\cdot}\boxed{\cdot\cdot}} \{1, \ldots, n\}$

# RSA-in-the-head (Maire-V. 2023)

$(N, e) \quad y = x^e \bmod N$





$x$ $\qquad\qquad\qquad\qquad$ $y$

$[\![x]\!] \xleftarrow{\boxed{\cdot\cdot}\boxed{\cdot\cdot}} [(\mathbb{Z}/N\mathbb{Z})^*]^n$

$(y_i \leftarrow x_i^e \bmod N)_i$

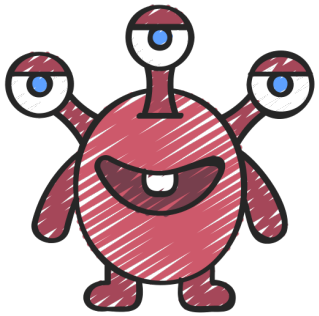
$\xrightarrow{\vec{y}}$

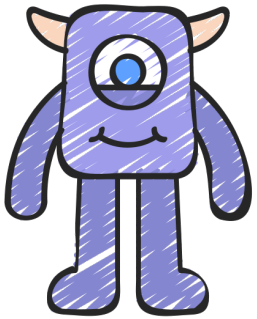
$i^\star \xleftarrow{\boxed{\cdot\cdot}\boxed{\cdot\cdot}} \{1, \ldots, n\}$

$\xleftarrow{i^\star}$

# RSA-in-the-head (Maire-V. 2023)

$$\boxed{(N, e) \quad y = x^e \bmod N}$$





$x$ $\qquad\qquad\qquad\qquad$ $y$

$$[\![x]\!] \xleftarrow{\boxed{\cdot}\,\boxed{\cdot\cdot}} [(\mathbb{Z}/N\mathbb{Z})^*]^n$$

$$(y_i \leftarrow x_i^e \bmod N)_i \xrightarrow{\vec{y}}$$

$$i^\star \xleftarrow{\boxed{\cdot}\,\boxed{\cdot\cdot}} \{1, \ldots, n\}$$

$$\xleftarrow{i^\star}$$

$$- \xrightarrow{(x_i)_{i \neq i^\star}}$$

$$x_i^e \stackrel{?}{\equiv} y_i \bmod N$$

$$y_1 \cdots y_n \stackrel{?}{\equiv} y \bmod N$$

# RSA-in-the-head

**Input:** An RSA integer $N$ and $y \in (\mathbb{Z}/N\mathbb{Z})^*$ - Alice **knows** $x$ s.t. $\boxed{x^e \equiv y \bmod N}$

1. Alice chooses $(n-1)$ random elements $x_1, \ldots, x_{n-1} \in (\mathbb{Z}/N\mathbb{Z})^*$ and sets $x_n = x/(x_1 \cdots x_{n-1}) \bmod N$.
2. She computes $y_i = (x_i^e \bmod N)$ for all $i \in \{1, \ldots, n\}$ and sends them to Bob,
3. Bob chooses randomly $i^* \in \{0, \ldots, n\}$ and sends it to Alice,
4. Alice reveals the values $x_i$ for all $i \in \{1, \ldots, n\}$ with $i \neq i^*$.
5. Bob checks
   1. if $y_i \equiv x_i^e \bmod N$ for all $i \in \{1, \ldots, n\}$ with $i \neq i^*$
   2. and if $y_1 \cdots y_n = y \bmod N$

# RSA-in-the-head

**Input:** An RSA integer $N$ and $y \in (\mathbb{Z}/N\mathbb{Z})^*$ - Alice **knows** $x$ s.t. $\boxed{x^e \equiv y \bmod N}$

1. Alice chooses $(n-1)$ random elements $x_1, \ldots, x_{n-1} \in (\mathbb{Z}/N\mathbb{Z})^*$ and sets $x_n = x/(x_1 \cdots x_{n-1}) \bmod N$.
2. She computes $y_i = (x_i^e \bmod N)$ for all $i \in \{1, \ldots, n\}$ and sends them to Bob,
3. Bob chooses randomly $i^* \in \{0, \ldots, n\}$ and sends it to Alice,
4. Alice reveals the values $x_i$ for all $i \in \{1, \ldots, n\}$ with $i \neq i^*$.
5. Bob checks
   1. if $y_i \equiv x_i^e \bmod N$ for all $i \in \{1, \ldots, n\}$ with $i \neq i^*$
   2. and if $y_1 \cdots y_n = y \bmod N$

# RSA-in-the-head

**Input:** An RSA integer $N$ and $y \in (\mathbb{Z}/N\mathbb{Z})^*$ - Alice **knows** $x$ s.t. $\boxed{x^e \equiv y \bmod N}$

1. Alice chooses $(n-1)$ random elements $x_1, \ldots, x_{n-1} \in (\mathbb{Z}/N\mathbb{Z})^*$ and sets $x_n = x/(x_1 \cdots x_{n-1}) \bmod N$.
2. She computes $y_i = (x_i^e \bmod N)$ for all $i \in \{1, \ldots, n\}$ and sends them to Bob,
3. Bob chooses randomly $i^* \in \{0, \ldots, n\}$ and sends it to Alice,
4. Alice reveals the values $x_i$ for all $i \in \{1, \ldots, n\}$ with $i \neq i^*$.
5. Bob checks
   1. if $y_i \equiv x_i^e \bmod N$ for all $i \in \{1, \ldots, n\}$ with $i \neq i^*$
   2. and if $y_1 \cdots y_n = y \bmod N$

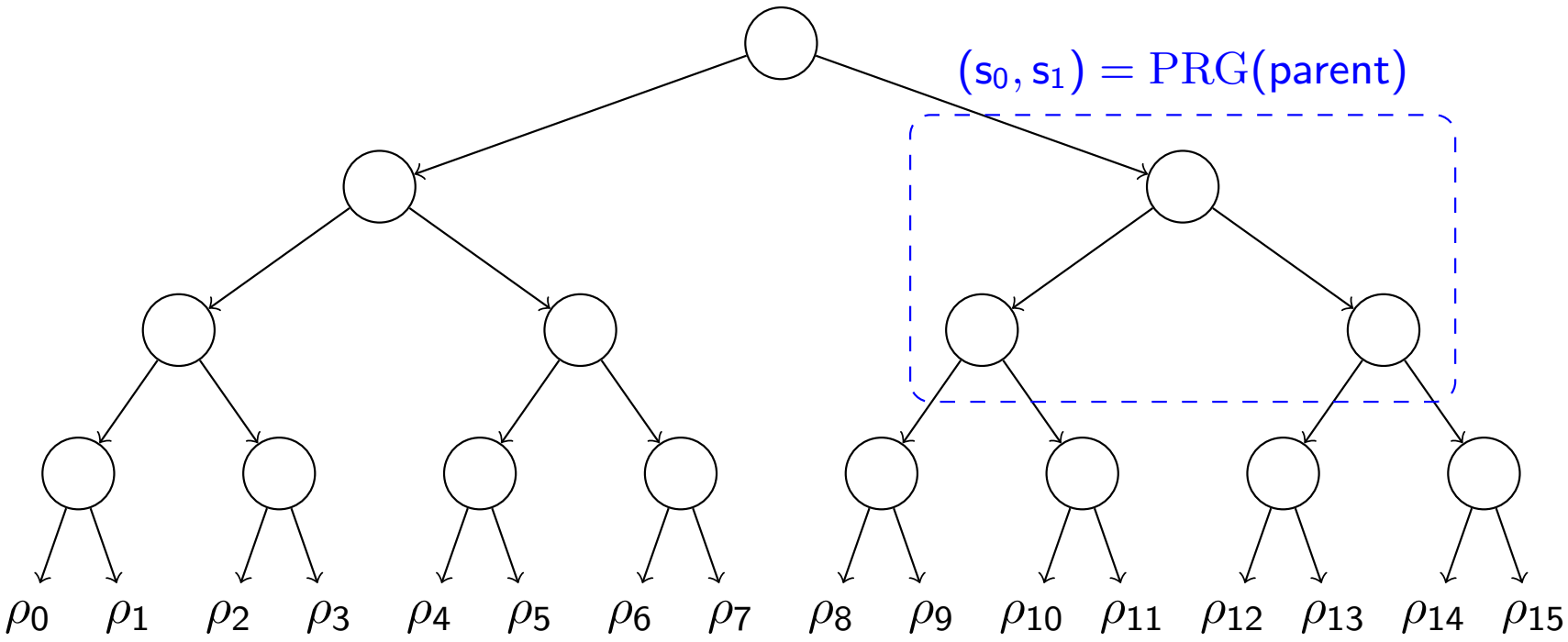
# RSA-in-the-head

**Input:** An RSA integer $N$ and $y \in (\mathbb{Z}/N\mathbb{Z})^*$ - Alice **knows** $x$ s.t. $\boxed{x^e \equiv y \bmod N}$

Repeat the following $\tau$ times

1. Alice chooses $(n-1)$ random elements $x_1, \ldots, x_{n-1} \in (\mathbb{Z}/N\mathbb{Z})^*$ and sets $x_n = x/(x_1 \cdots x_{n-1}) \bmod N$.
2. She computes $y_i = (x_i^e \bmod N)$ for all $i \in \{1, \ldots, n\}$ and sends them to Bob,
3. Bob chooses randomly $i^* \in \{0, \ldots, n\}$ and sends it to Alice,
4. Alice reveals the values $x_i$ for all $i \in \{1, \ldots, n\}$ with $i \neq i^*$.
5. Bob checks
   1. if $y_i \equiv x_i^e \bmod N$ for all $i \in \{1, \ldots, n\}$ with $i \neq i^*$
   2. and if $y_1 \cdots y_n = y \bmod N$

# Compressing the Proof - Tree PRG




**Commitment**

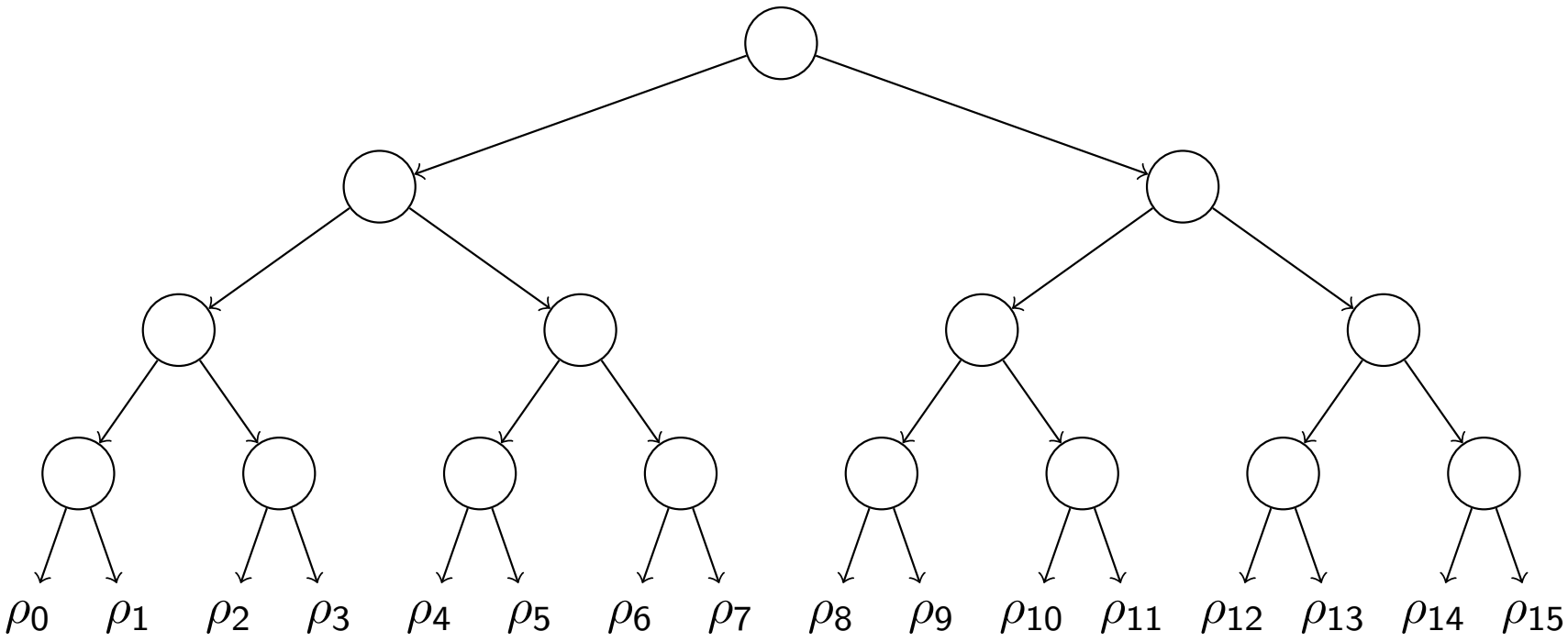
1. $(x_i, r_i) = \mathrm{PRG}(\rho_i)$
2. $r_n, r$ picked at random
3. $c_i = H(y_i, r_i)$
4. $c = H(c_0, \ldots, c_n, r)$ and $\Delta_x$

**Response**

Alice reveals $r$ and

1. $\log_2(n)$ values in the tree (in blue)
2. $c_{i^*}$

# Compressing the Proof - Tree PRG




**Commitment**

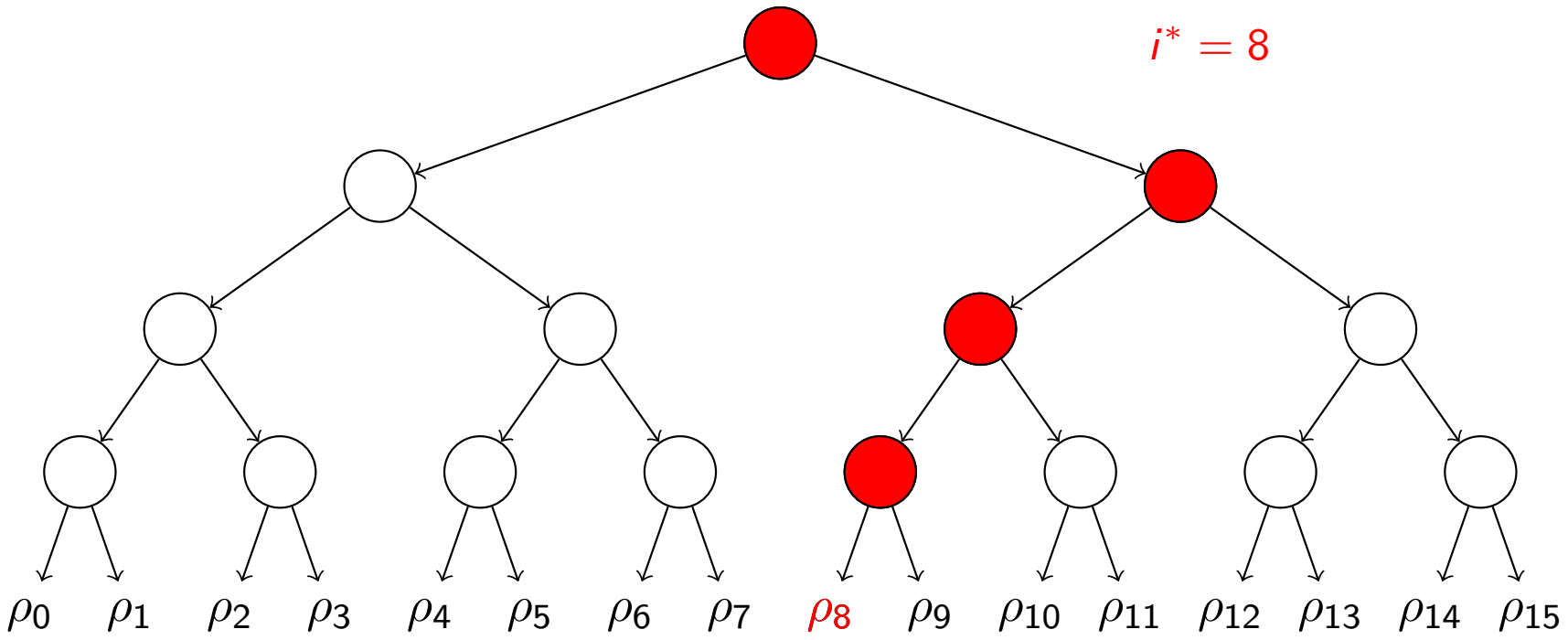
1. $(x_i, r_i) = \mathrm{PRG}(\rho_i)$
2. $r_n, r$ picked at random
3. $c_i = H(y_i, r_i)$
4. $c = H(c_0, \ldots, c_n, r)$ and $\Delta_x$

**Response**

Alice reveals $r$ and

1. $\log_2(n)$ values in the tree (in blue)
2. $c_{i^*}$

# Compressing the Proof - Tree PRG




**Commitment**

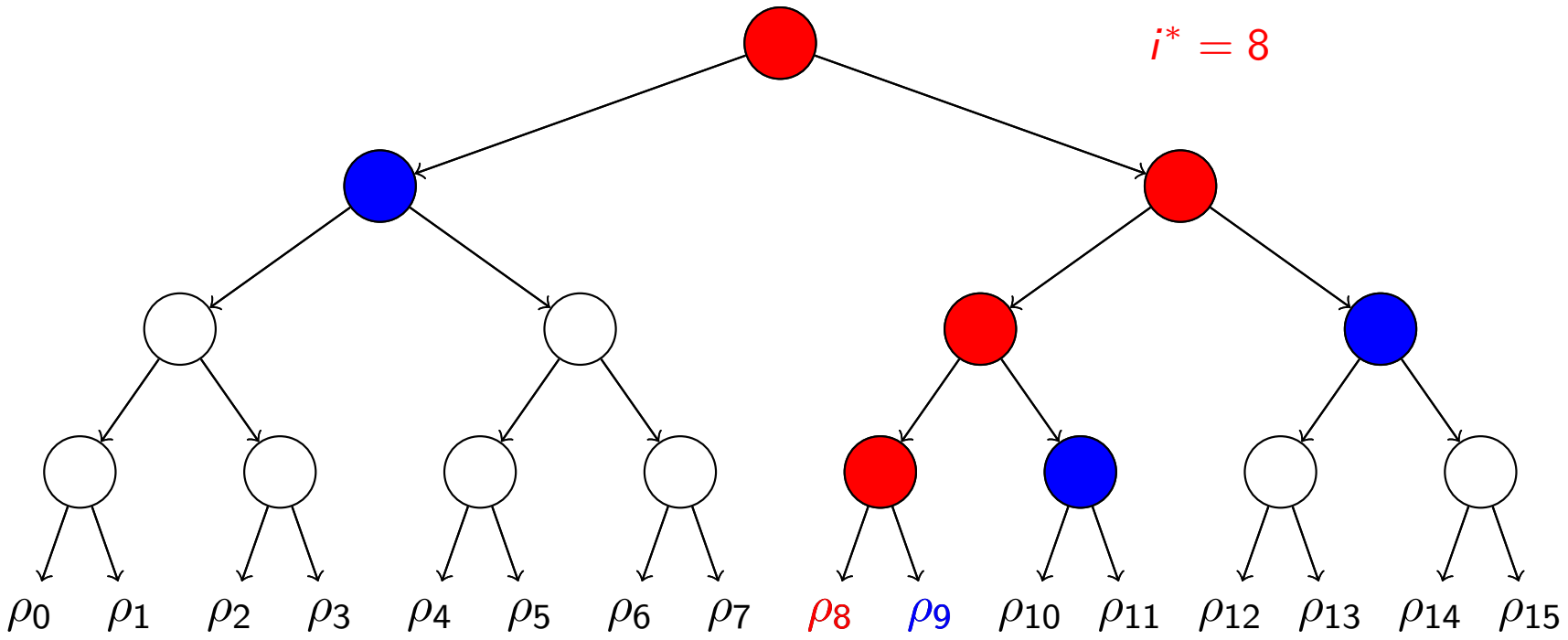
1. $(x_i, r_i) = \mathrm{PRG}(\rho_i)$
2. $r_n, r$ picked at random
3. $c_i = H(y_i, r_i)$
4. $c = H(c_0, \ldots, c_n, r)$ and $\Delta_x$

**Response**

Alice reveals $r$ and

1. $\log_2(n)$ values in the tree (in blue)
2. $c_{i^*}$

# Compressing the Proof - Tree PRG




**Commitment**

1. $(x_i, r_i) = \mathrm{PRG}(\rho_i)$
2. $r_n, r$ picked at random
3. $c_i = H(y_i, r_i)$
4. $c = H(c_0, \ldots, c_n, r)$ and $\Delta_x$

**Response**

Alice reveals $r$ and

1. $\log_2(n)$ values in the tree (in blue)
2. $c_{i^*}$

# RSA-in-the-head

$$\boxed{e = 3,\ N \simeq 2^{2048},\ \lambda = 128}$$

1. Guillou-Quisquater
   - Soundness error: $1/e = 1/3$
   - Iterations: 80
   - Proof size: $80 \times 2048 + 256 = 20.5$ KBytes

2. RSA-in-the-head
   - Soundness error: $1/n = 1/256$
   - Iterations: 16
   - Proof size: $16 \times (8 \times 128 + 2048) + 256 + 128 = 6.5$ KBytes

# RSA-in-the-head

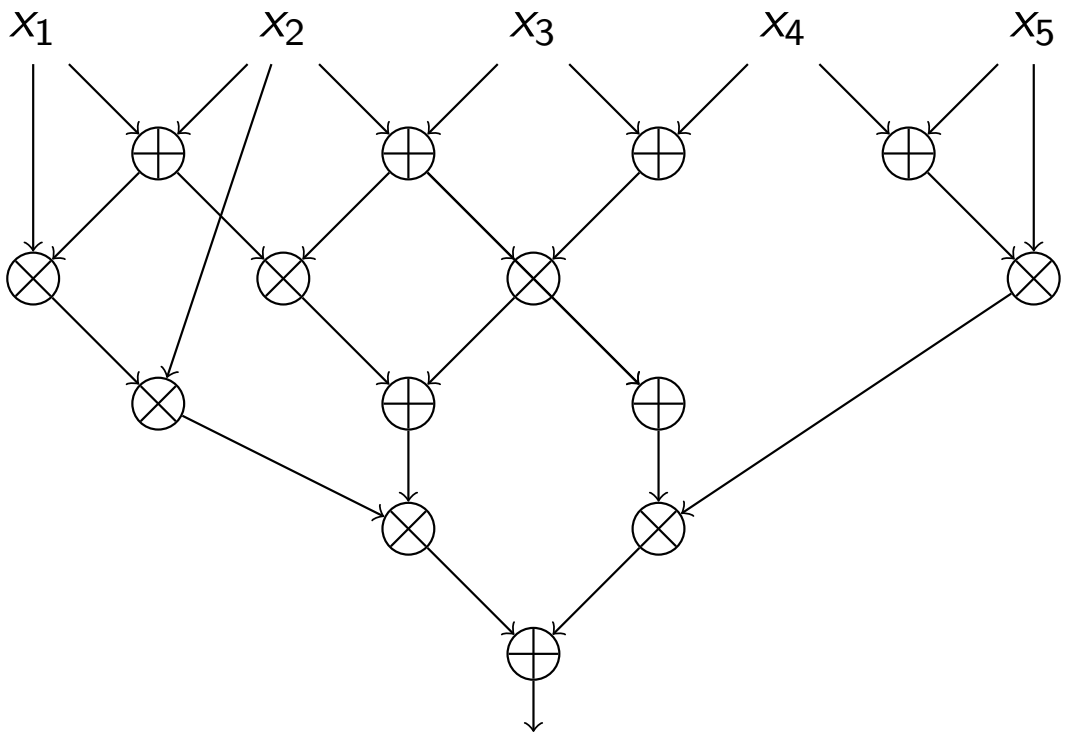
$$\boxed{e = 17,\ N \simeq 2^{2048},\ \lambda = 128}$$

1. Guillou-Quisquater
   - Soundness error: $1/e = 1/17$
   - Iterations: 32
   - Proof size: $32 \times 2048 + 256 = 8.2$ KBytes

2. RSA-in-the-head
   - Soundness error: $1/n = 1/256$
   - Iterations: 16
   - Proof size: $16 \times (8 \times 128 + 2048) + 256 + 128 = 6.5$ KBytes

# Beyond Linear functions?

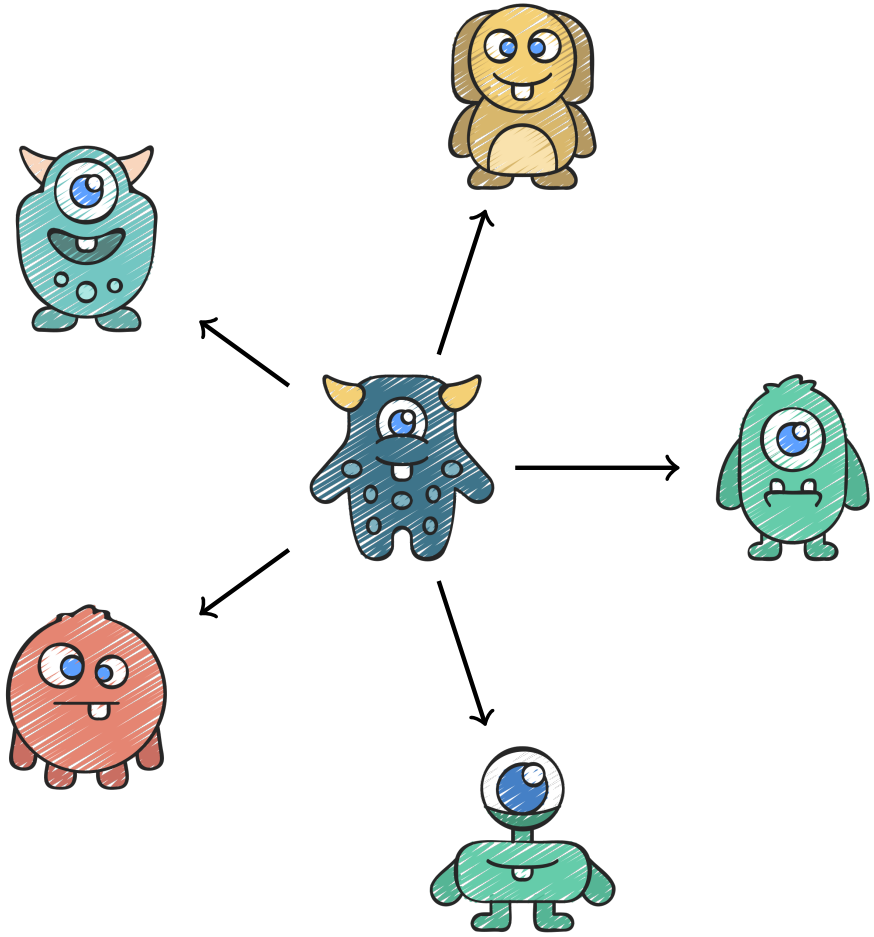
- use **additive sharing** ($n$-out-of-$n$) in a finite field $\mathbb{F}$
- represent $f$ using an arithmetic circuit over $\mathbb{F}$




$f(x_1, x_2, x_3, x_4, x_5)$

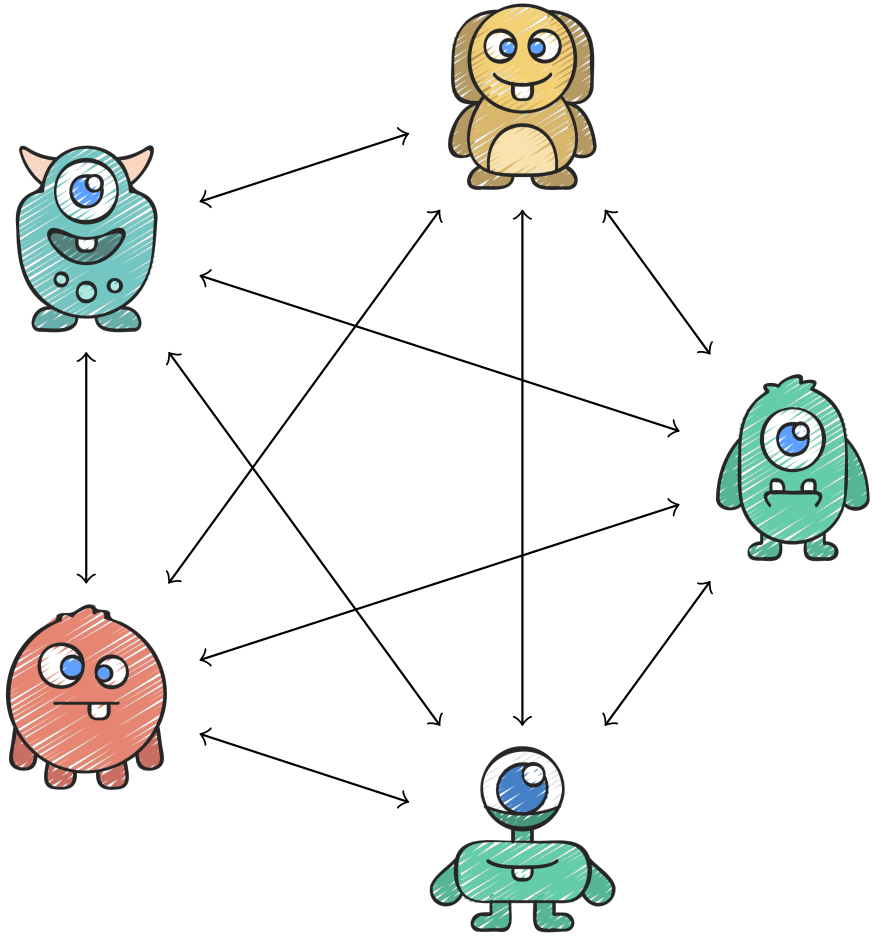
- linear gates are "easy"
- **How** to handle multiplication gates?

# MPC with Pre-Processing



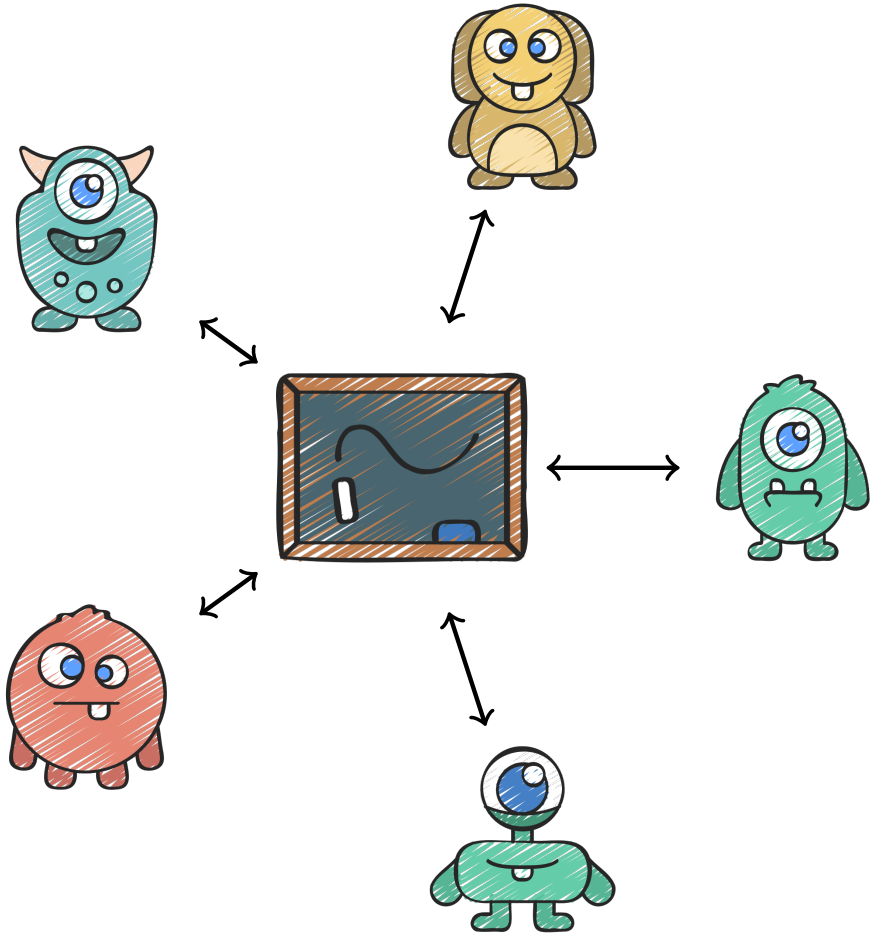
- parties obtain correlated secret inputs
- pre-processing is input independent

# MPC with Pre-Processing



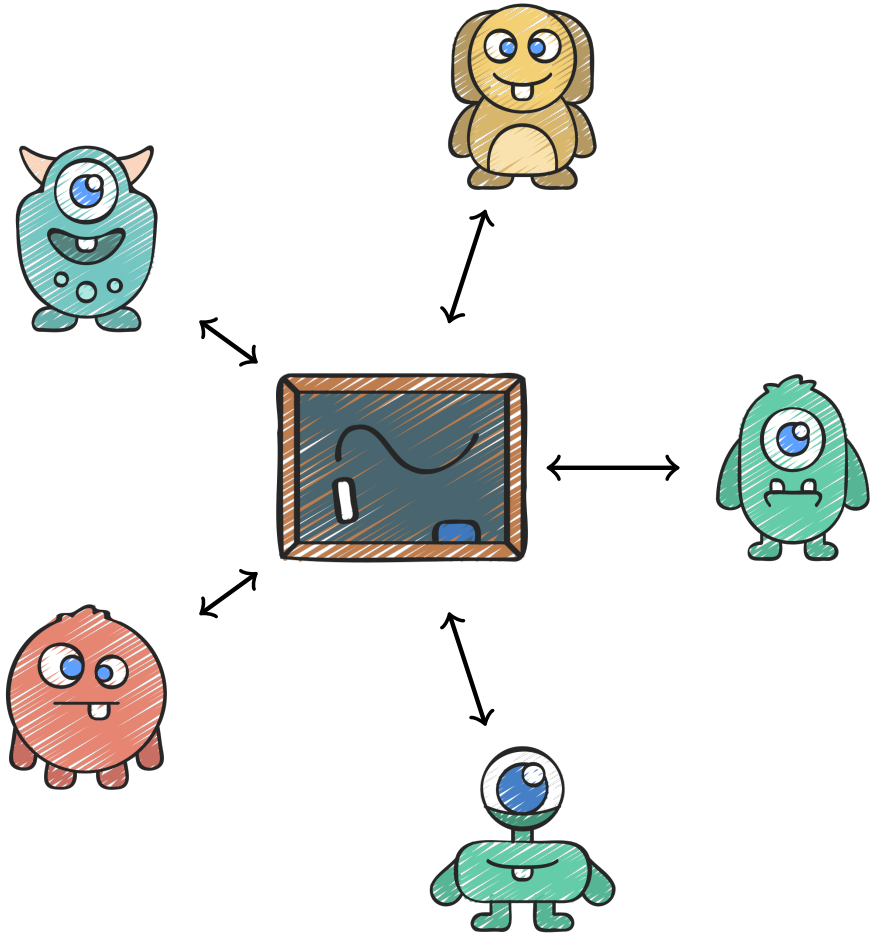
- parties obtain correlated secret inputs
- pre-processing is input independent
- parties then run the MPC protocol

# MPC with Pre-Processing



- parties obtain correlated secret inputs
- pre-processing is input independent
- parties then run the MPC protocol
- lowers the cost (broadcast only)

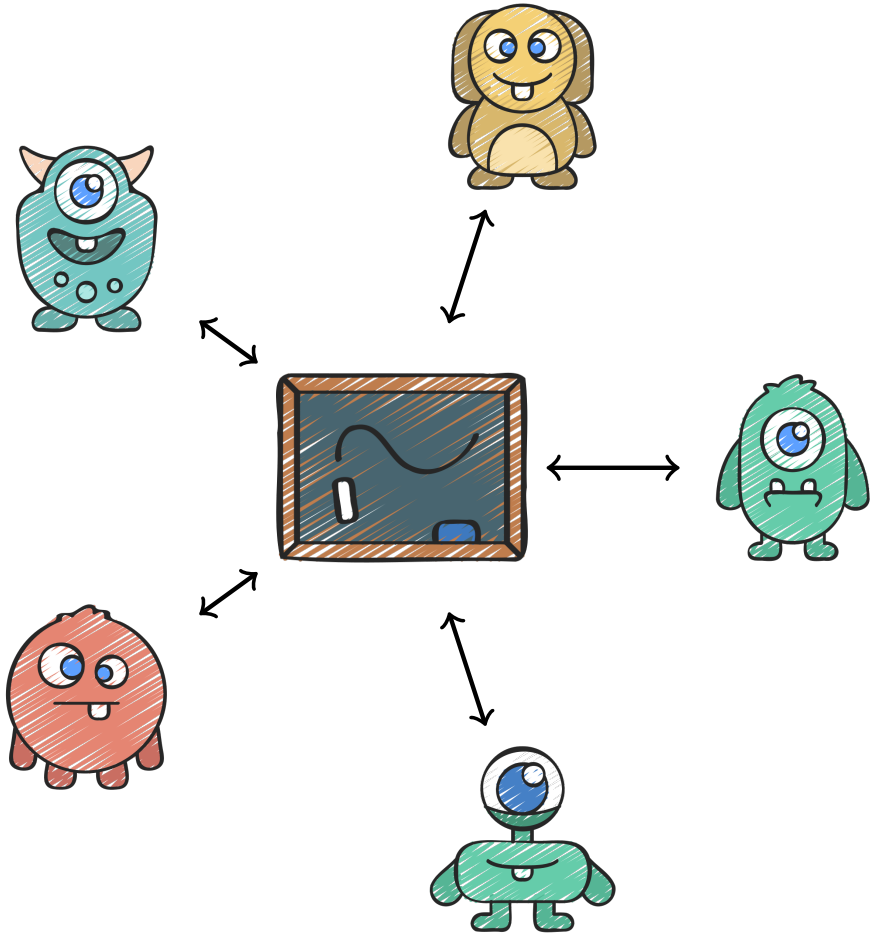
# MPC with Pre-Processing



- parties obtain correlated secret inputs
- pre-processing is input independent
- parties then run the MPC protocol
- lowers the cost (broadcast only)

- **How to use it in MPC in the head?**

# MPC with Pre-Processing



- parties obtain correlated secret inputs
- pre-processing is input independent
- parties then run the MPC protocol
- lowers the cost (broadcast only)

- **How to use it in MPC in the head?**
- ...gives Alice more opportunities to cheat!

# Beaver Triples

**Beaver – Crypto 1991**

$$[\![x]\!] = \textsc{Share}(x) = (x_1, \ldots, x_n) \quad [\![y]\!] = \textsc{Share}(y) = (y_1, \ldots, y_n)$$

- Given $[\![a]\!]$, $[\![b]\!]$ and $[\![c]\!]$ where $a$ and $b$ are random and $c = a \cdot b$
- $P_i$ computes $\alpha_i = x_i - a_i$ and $\beta_i = y_i - b_i$ and broadcasts them

$$\alpha = \alpha_1 + \cdots + \alpha_n = x - a \qquad \beta = \beta_1 + \cdots + \beta_n = y - b$$

- We have

$$\alpha \cdot \beta + \beta \cdot a + \alpha \cdot b + c = xy$$

- $P_i$ computes $z_i = \alpha \cdot \beta + \beta \cdot a_i + \alpha \cdot b_i + c_i$

$$[\![z]\!] = (z_1, \ldots, z_n) \simeq \textsc{Share}(x \cdot y)$$

# Beaver Triples

Beaver – Crypto 1991

$$[\![x]\!] = \textsc{Share}(x) = (x_1, \ldots, x_n) \quad [\![y]\!] = \textsc{Share}(y) = (y_1, \ldots, y_n)$$

- Given $[\![a]\!]$, $[\![b]\!]$ and $[\![c]\!]$ where $a$ and $b$ are random and $c = a \cdot b$
- $P_i$ computes $\alpha_i = x_i - a_i$ and $\beta_i = y_i - b_i$ and broadcasts them

$$\alpha = \alpha_1 + \cdots + \alpha_n = x - a \qquad \beta = \beta_1 + \cdots + \beta_n = y - b$$

- We have

$$\alpha \cdot \beta + \beta \cdot a + \alpha \cdot b + c = xy$$

- $P_i$ computes $z_i = \alpha \cdot \beta + \beta \cdot a_i + \alpha \cdot b_i + c_i$

$$[\![z]\!] = (z_1, \ldots, z_n) \simeq \textsc{Share}(x \cdot y)$$

# Beaver Triples

Beaver – Crypto 1991

$$[\![x]\!] = \textsc{Share}(x) = (x_1, \ldots, x_n) \quad [\![y]\!] = \textsc{Share}(y) = (y_1, \ldots, y_n)$$

- Given $[\![a]\!]$, $[\![b]\!]$ and $[\![c]\!]$ where $a$ and $b$ are random and $c = a \cdot b$
- $P_i$ computes $\alpha_i = x_i - a_i$ and $\beta_i = y_i - b_i$ and broadcasts them

$$\alpha = \alpha_1 + \cdots + \alpha_n = x - a \qquad \beta = \beta_1 + \cdots + \beta_n = y - b$$

- We have

$$\alpha \cdot \beta + \beta \cdot a + \alpha \cdot b + c = xy$$

- $P_i$ computes $z_i = \alpha \cdot \beta + \beta \cdot a_i + \alpha \cdot b_i + c_i$

$$[\![z]\!] = (z_1, \ldots, z_n) \simeq \textsc{Share}(x \cdot y)$$

# Beaver Triples

Beaver – Crypto 1991

$$[\![x]\!] = \textsc{Share}(x) = (x_1, \ldots, x_n) \quad [\![y]\!] = \textsc{Share}(y) = (y_1, \ldots, y_n)$$

- Given $[\![a]\!]$, $[\![b]\!]$ and $[\![c]\!]$ where $a$ and $b$ are random and $c = a \cdot b$
- $P_i$ computes $\alpha_i = x_i - a_i$ and $\beta_i = y_i - b_i$ and broadcasts them

$$\alpha = \alpha_1 + \cdots + \alpha_n = x - a \qquad \beta = \beta_1 + \cdots + \beta_n = y - b$$

- We have

$$\alpha \cdot \beta + \beta \cdot a + \alpha \cdot b + c = xy$$

- $P_i$ computes $z_i = \alpha \cdot \beta + \beta \cdot a_i + \alpha \cdot b_i + c_i$

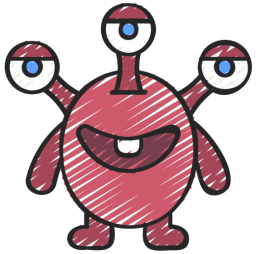
$$[\![z]\!] = (z_1, \ldots, z_n) \simeq \textsc{Share}(x \cdot y)$$

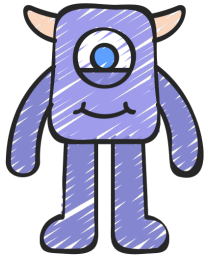
# MPC in the Head with Beaver Triples

Katz, Kolesnikov, Wang – CCS 2018

$([\![a_i]\!], [\![b_i]\!], [\![c_i]\!]) \xleftarrow{\text{⚀⚅}}$ **Triples**

$$\xrightarrow{\{C_i = \textsc{Commit}([\![a_i]\!], [\![b_i]\!], [\![c_i]\!])\}_{i \in S}}$$





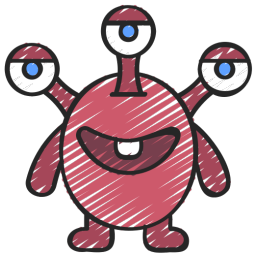
# MPC in the Head with Beaver Triples

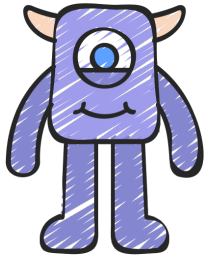
Katz, Kolesnikov, Wang – CCS 2018

$([\![a_i]\!], [\![b_i]\!], [\![c_i]\!]) \xleftarrow{\text{⚀⚁}}$ **Triples**

$$\xrightarrow{\{C_i = \textsc{Commit}([\![a_i]\!], [\![b_i]\!], [\![c_i]\!])\}_{i \in S}}$$

$$\xleftarrow{T \subset S}$$





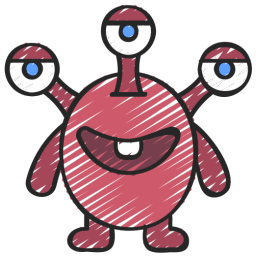
# MPC in the Head with Beaver Triples

Katz, Kolesnikov, Wang – CCS 2018

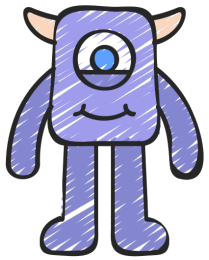
$([\![a_i]\!], [\![b_i]\!], [\![c_i]\!]) \xleftarrow{\text{⚀⚁}}$ **Triples**

$\{C_i = \textsc{Commit}([\![a_i]\!], [\![b_i]\!], [\![c_i]\!])\}_{i \in S}$ $\longrightarrow$

$T \subset S$ $\longleftarrow$



$\{\textsc{Open}(C_i)\}_{i \in T}$ $\longrightarrow$



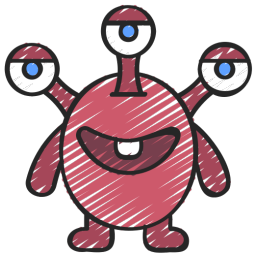
# MPC in the Head with Beaver Triples

Katz, Kolesnikov, Wang – CCS 2018

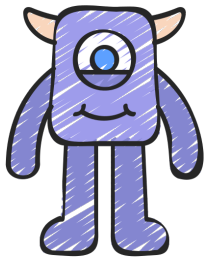
$([\![a_i]\!], [\![b_i]\!], [\![c_i]\!]) \xleftarrow{\boxed{\cdot\cdot}\boxed{\cdot\cdot}}$ **Triples**

$\xrightarrow{\{C_i = \textsc{Commit}([\![a_i]\!], [\![b_i]\!], [\![c_i]\!])\}_{i \in S}}$

$\xleftarrow{T \subset S}$



$\xrightarrow{\{\textsc{Open}(C_i)\}_{i \in T}}$



Run MPC with triples in $S \setminus T$

$\downarrow$

$(\mathrm{View}_1, \ldots, \mathrm{View}_n)$

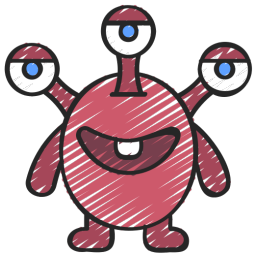
$\xrightarrow{\{C'_j = \textsc{Commit}(\mathrm{View}_j)\}_{1 \le j \le n}}$

# MPC in the Head with Beaver Triples

Katz, Kolesnikov, Wang – CCS 2018

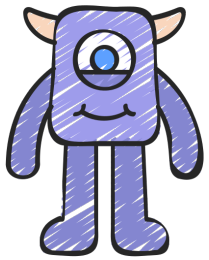
$([\![a_i]\!], [\![b_i]\!], [\![c_i]\!]) \xleftarrow{\boxed{\cdot\cdot}\boxed{\cdot\cdot}}$ **Triples**

$\xrightarrow{\{C_i = \textsc{Commit}([\![a_i]\!], [\![b_i]\!], [\![c_i]\!])\}_{i \in S}}$



$\xleftarrow{T \subset S}$

$\xrightarrow{\{\textsc{Open}(C_i)\}_{i \in T}}$



Run MPC with triples in $S \setminus T$

$\downarrow$

$(\mathrm{View}_1, \ldots, \mathrm{View}_n)$

$\xrightarrow{\{C'_j = \textsc{Commit}(\mathrm{View}_j)\}_{1 \le j \le n}}$

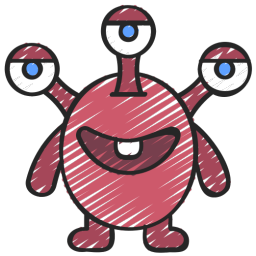
$\xleftarrow{j^\star}$

# MPC in the Head with Beaver Triples

Katz, Kolesnikov, Wang – CCS 2018

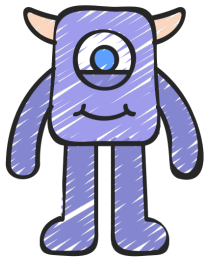
$([\![a_i]\!], [\![b_i]\!], [\![c_i]\!]) \xleftarrow{\text{⚀⚁}}$ **Triples**

$\xrightarrow{\{C_i = \textsc{Commit}([\![a_i]\!], [\![b_i]\!], [\![c_i]\!])\}_{i \in S}}$



$\xleftarrow{T \subset S}$

$\xrightarrow{\{\textsc{Open}(C_i)\}_{i \in T}}$



Run MPC with triples in $S \setminus T$

$\downarrow$

$(\mathrm{View}_1, \ldots, \mathrm{View}_n)$

$\xrightarrow{\{C'_j = \textsc{Commit}(\mathrm{View}_j)\}_{1 \le j \le n}}$

$\xleftarrow{j^\star}$

$\xrightarrow{\{\textsc{Open}(C_j)_{j \neq j^\star}\}}$

# Verifying is Cheaper than Computing . . .

- $|S \setminus T|$ = number of multiplication gates
- $S$ has to be large enough to detect a cheating Alice with high probability (e.g. for $n = 16$, $\simeq \times 10$ overhead)

- **Idea:** Replace computing $x \cdot y$ from their sharing by committing $z = x \cdot y$ and checking that $z$ is correct
- "sacrifice" a committed triple $(a, b, c)$ with $c = a \cdot b$ (that is checked simultaneously)
  $\rightsquigarrow$ no need for **'cut and choose"**

Baum, Nof – PKC 2020

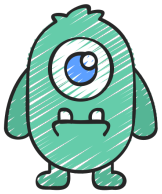
# Verifying is Cheaper than Computing . . .

- $|S \setminus T|$ = number of multiplication gates
- $S$ has to be large enough to detect a cheating Alice with high probability (e.g. for $n = 16$, $\simeq \times 10$ overhead)

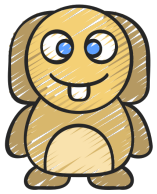
- **Idea:** Replace computing $x \cdot y$ from their sharing by committing $z = x \cdot y$ and checking that $z$ is correct
- "sacrifice" a committed triple $(a, b, c)$ with $c = a \cdot b$ (that is checked simultaneously)
  $\rightsquigarrow$ no need for **'cut and choose"**

Baum, Nof – PKC 2020

# Checking Products

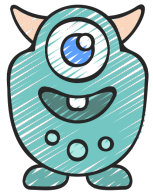


$[\![x]\!]_1, [\![y]\!]_1, [\![z]\!]_1$
$[\![a]\!]_1, [\![b]\!]_1, [\![c]\!]_1$



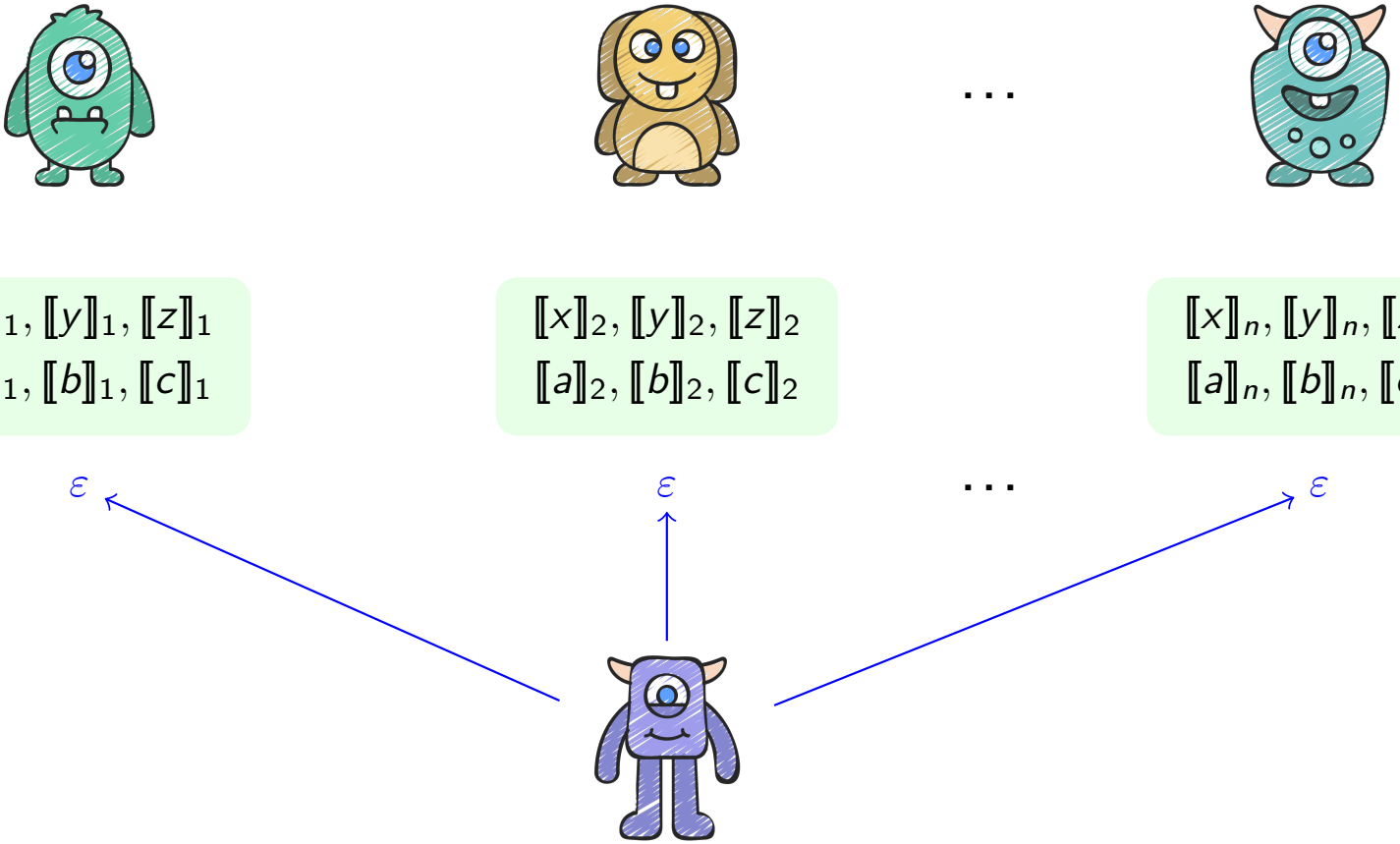
$[\![x]\!]_2, [\![y]\!]_2, [\![z]\!]_2$
$[\![a]\!]_2, [\![b]\!]_2, [\![c]\!]_2$

...

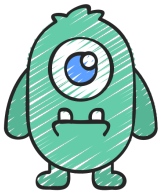


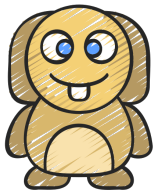
$[\![x]\!]_n, [\![y]\!]_n, [\![z]\!]_n$
$[\![a]\!]_n, [\![b]\!]_n, [\![c]\!]_n$

# Checking Products

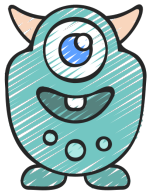


# Checking Products





$\cdots$



$[\![x]\!]_1, [\![y]\!]_1, [\![z]\!]_1$
$[\![a]\!]_1, [\![b]\!]_1, [\![c]\!]_1$

$[\![x]\!]_2, [\![y]\!]_2, [\![z]\!]_2$
$[\![a]\!]_2, [\![b]\!]_2, [\![c]\!]_2$

$[\![x]\!]_n, [\![y]\!]_n, [\![z]\!]_n$
$[\![a]\!]_n, [\![b]\!]_n, [\![c]\!]_n$

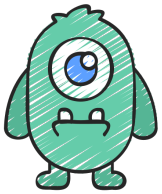
$\varepsilon$ $\quad$ $\varepsilon$ $\quad$ $\cdots$ $\quad$ $\varepsilon$

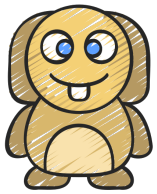
$[\![\alpha]\!]_1 = \varepsilon[\![x]\!]_1 + [\![a]\!]_1$
$[\![\beta]\!]_1 = [\![y]\!]_1 + [\![b]\!]_1$

$[\![\alpha]\!]_2 = \varepsilon[\![x]\!]_2 + [\![a]\!]_2$
$[\![\beta]\!]_2 = [\![y]\!]_2 + [\![b]\!]_2$

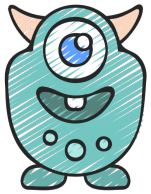
$[\![\alpha]\!]_n = \varepsilon[\![x]\!]_n + [\![a]\!]_n$
$[\![\beta]\!]_n = [\![y]\!]_n + [\![b]\!]_n$

# Checking Products





$\cdots$



$[\![x]\!]_1, [\![y]\!]_1, [\![z]\!]_1$
$[\![a]\!]_1, [\![b]\!]_1, [\![c]\!]_1$

$[\![x]\!]_2, [\![y]\!]_2, [\![z]\!]_2$
$[\![a]\!]_2, [\![b]\!]_2, [\![c]\!]_2$

$[\![x]\!]_n, [\![y]\!]_n, [\![z]\!]_n$
$[\![a]\!]_n, [\![b]\!]_n, [\![c]\!]_n$

$\varepsilon$ $\quad$ $\varepsilon$ $\quad$ $\cdots$ $\quad$ $\varepsilon$

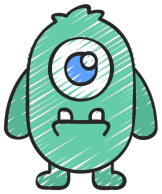
$[\![\alpha]\!]_1 = \varepsilon[\![x]\!]_1 + [\![a]\!]_1$
$[\![\beta]\!]_1 = [\![y]\!]_1 + [\![b]\!]_1$

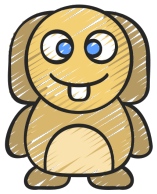
$[\![\alpha]\!]_2 = \varepsilon[\![x]\!]_2 + [\![a]\!]_2$
$[\![\beta]\!]_2 = [\![y]\!]_2 + [\![b]\!]_2$

$[\![\alpha]\!]_n = \varepsilon[\![x]\!]_n + [\![a]\!]_n$
$[\![\beta]\!]_n = [\![y]\!]_n + [\![b]\!]_n$

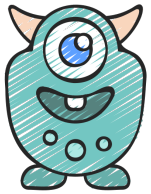
$[\![\alpha]\!]_1, [\![\beta]\!]_2$ $\quad$ $[\![\alpha]\!]_2, [\![\beta]\!]_2$ $\quad$ $\cdots$ $\quad$ $[\![\alpha]\!]_n, [\![\beta]\!]_n$

# Checking Products





$\cdots$



$[\![x]\!]_1, [\![y]\!]_1, [\![z]\!]_1$
$[\![a]\!]_1, [\![b]\!]_1, [\![c]\!]_1$

$[\![x]\!]_2, [\![y]\!]_2, [\![z]\!]_2$
$[\![a]\!]_2, [\![b]\!]_2, [\![c]\!]_2$

$[\![x]\!]_n, [\![y]\!]_n, [\![z]\!]_n$
$[\![a]\!]_n, [\![b]\!]_n, [\![c]\!]_n$

$\varepsilon$ $\quad$ $\varepsilon$ $\quad \cdots \quad$ $\varepsilon$

$[\![\alpha]\!]_1 = \varepsilon[\![x]\!]_1 + [\![a]\!]_1$
$[\![\beta]\!]_1 = [\![y]\!]_1 + [\![b]\!]_1$

$[\![\alpha]\!]_2 = \varepsilon[\![x]\!]_2 + [\![a]\!]_2$
$[\![\beta]\!]_2 = [\![y]\!]_2 + [\![b]\!]_2$

$[\![\alpha]\!]_n = \varepsilon[\![x]\!]_n + [\![a]\!]_n$
$[\![\beta]\!]_n = [\![y]\!]_n + [\![b]\!]_n$

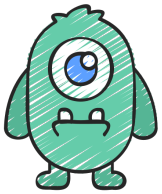
$[\![\alpha]\!]_1, [\![\beta]\!]_2$ $\quad$ $[\![\alpha]\!]_2, [\![\beta]\!]_2$ $\quad \cdots \quad$ $[\![\alpha]\!]_n, [\![\beta]\!]_n$

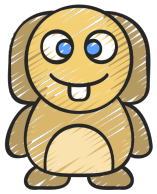
$[\![v]\!]_1 = \varepsilon[\![z]\!]_1 + \alpha[\![b]\!]_1$
$-[\![c]\!]_1 + \beta[\![a]\!]_1 - \alpha\beta$

$[\![v]\!]_2 = \varepsilon[\![z]\!]_2 + \alpha[\![b]\!]_2$
$-[\![c]\!]_2 + \beta[\![a]\!]_2$

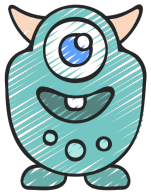
$[\![v]\!]_n = \varepsilon[\![z]\!]_n + \alpha[\![b]\!]_n$
$-[\![c]\!]_n + \beta[\![a]\!]_n$

# Checking Products





$\cdots$



$[\![x]\!]_1, [\![y]\!]_1, [\![z]\!]_1$
$[\![a]\!]_1, [\![b]\!]_1, [\![c]\!]_1$

$[\![x]\!]_2, [\![y]\!]_2, [\![z]\!]_2$
$[\![a]\!]_2, [\![b]\!]_2, [\![c]\!]_2$

$[\![x]\!]_n, [\![y]\!]_n, [\![z]\!]_n$
$[\![a]\!]_n, [\![b]\!]_n, [\![c]\!]_n$

$\varepsilon$ $\quad$ $\varepsilon$ $\quad \cdots \quad$ $\varepsilon$

$$[\![\alpha]\!]_1 = \varepsilon[\![x]\!]_1 + [\![a]\!]_1$$
$$[\![\beta]\!]_1 = [\![y]\!]_1 + [\![b]\!]_1$$

$$[\![\alpha]\!]_2 = \varepsilon[\![x]\!]_2 + [\![a]\!]_2$$
$$[\![\beta]\!]_2 = [\![y]\!]_2 + [\![b]\!]_2$$

$$[\![\alpha]\!]_n = \varepsilon[\![x]\!]_n + [\![a]\!]_n$$
$$[\![\beta]\!]_n = [\![y]\!]_n + [\![b]\!]_n$$

$[\![\alpha]\!]_1, [\![\beta]\!]_2$ $\quad$ $[\![\alpha]\!]_2, [\![\beta]\!]_2$ $\quad \cdots \quad$ $[\![\alpha]\!]_n, [\![\beta]\!]_n$

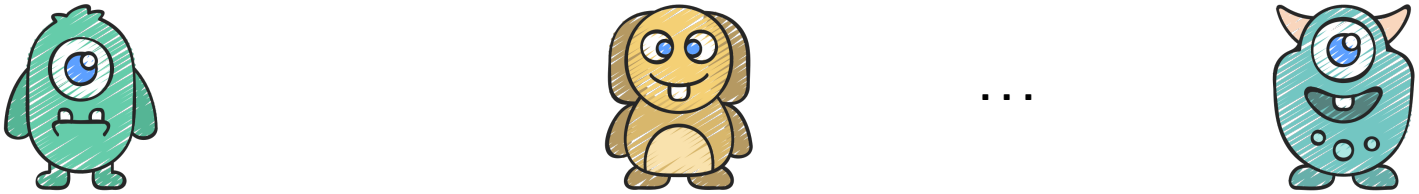
$$[\![v]\!]_1 = \varepsilon[\![z]\!]_1 + \alpha[\![b]\!]_1 - [\![c]\!]_1 + \beta[\![a]\!]_1 - \alpha\beta$$

$$[\![v]\!]_2 = \varepsilon[\![z]\!]_2 + \alpha[\![b]\!]_2 - [\![c]\!]_2 + \beta[\![a]\!]_2$$

$$[\![v]\!]_n = \varepsilon[\![z]\!]_n + \alpha[\![b]\!]_n - [\![c]\!]_n + \beta[\![a]\!]_n$$

$[\![v]\!]_1$ $\quad$ $[\![v]\!]_2$ $\quad \cdots \quad$ $[\![v]\!]_n$

# Checking Products



$[\![x]\!]_1, [\![y]\!]_1,$ $[\![x]\!]_n, [\![y]\!]_n, [\![z]\!]_n$

$[\![a]\!]_1, [\![b]\!]_1,$ $[\![a]\!]_n, [\![b]\!]_n, [\![c]\!]_n$

$\varepsilon$ $\varepsilon$

$[\![\alpha]\!]_1 = \varepsilon[\![x]\!]_1$ $]\!]_n = \varepsilon[\![x]\!]_n + [\![a]\!]_n$

$[\![\beta]\!]_1 = [\![y]\!]_1$ $]\!]_n = [\![y]\!]_n + [\![b]\!]_n$

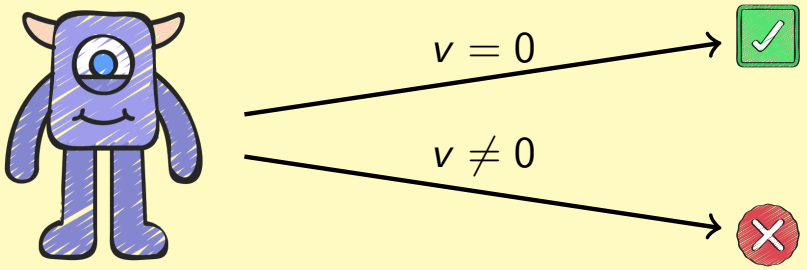
$[\![\alpha]\!]_1, [\![\beta]$ $[\![\alpha]\!]_n, [\![\beta]\!]_n$

$[\![v]\!]_1 = \varepsilon[\![z]\!]_1$ $]\!]_n = \varepsilon[\![z]\!]_n + \alpha[\![b]\!]_n$

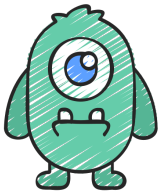
$-[\![c]\!]_1 + \beta[\![a]\!]_1 - \alpha\beta$ $-[\![c]\!]_2 + \beta[\![a]\!]_2$ $-[\![c]\!]_n + \beta[\![a]\!]_n$

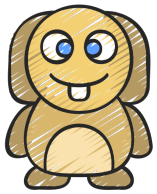
$[\![v]\!]_1$ $[\![v]\!]_2$ $\cdots$ $[\![v]\!]_n$

$$v = \varepsilon(xy - z) + (ab - c)$$

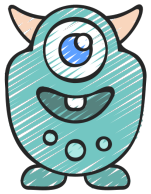


# Checking Products





...



$[\![x]\!]_1, [\![y]\!]_1,$ $[\![x]\!]_n, [\![y]\!]_n, [\![z]\!]_n$

$[\![a]\!]_1, [\![b]\!]_1,$ $[\![a]\!]_n, [\![b]\!]_n, [\![c]\!]_n$

$\varepsilon$ $\varepsilon$

$[\![\alpha]\!]_1 = \varepsilon[\![x]\!]_1$ $]\!]_n = \varepsilon[\![x]\!]_n + [\![a]\!]_n$

$[\![\beta]\!]_1 = [\![y]\!]_1$ $]\!]_n = [\![y]\!]_n + [\![b]\!]_n$

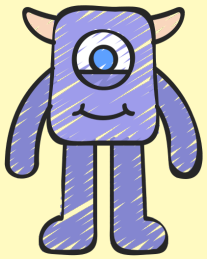
$[\![\alpha]\!]_1, [\![\beta]$ $[\![\alpha]\!]_n, [\![\beta]\!]_n$

$[\![v]\!]_1 = \varepsilon[\![z]\!]_1$ $]\!]_n = \varepsilon[\![z]\!]_n + \alpha[\![b]\!]_n$

$-[\![c]\!]_1 + \beta[\![a]\!]_1 - \alpha\beta$ $-[\![c]\!]_2 + \beta[\![a]\!]_2$ $-[\![c]\!]_n + \beta[\![a]\!]_n$

$[\![v]\!]_1$ $[\![v]\!]_2$ ... $[\![v]\!]_n$

$$v = \varepsilon(xy - z) + (ab - c)$$



$v = 0$ → ✓

$v \neq 0$ → ✗

**Soundness error:** $1/|\mathbb{F}|$

# Example: Subset-Sum

Given $(w_1, \ldots, w_\ell, t) \in (\mathbb{Z}/p\mathbb{Z})^{\ell+1}$, find $(x_1, \ldots, x_\ell) \in \{0,1\}^\ell$ s.t.

$$w_1 \cdot x_1 + \cdots + w_\ell \cdot x_\ell = t \bmod p$$

- **Linear relation:** $w_1 \cdot x_1 + \cdots + w_\ell \cdot x_\ell = t \bmod p$
- $x_i \in \{0,1\} \xrightarrow{\text{Arithmetization}} x_i(x_i - 1) = 0 \bmod p$
  $\rightsquigarrow \ell$ triples $\rightsquigarrow 2\ell$ auxiliary values + Tree PRG
- For $\ell = [\log_2(p)] = 256$, $n = 256 \rightsquigarrow 264$ KB!

| | |
|---|---|
| Shamir – Unpublished, 1986 | 1186 KB |
| Ling, Nguyen, Stehlé, Wang – PKC 2013 | 2350 KB |
| Beullens – Eurocrypt 2020 | 122 KB |
| Feneuil, Maire, Rivain, V. – Asiacrypt 2022 | 16 KB |

# Example: Subset-Sum

Given $(w_1, \ldots, w_\ell, t) \in (\mathbb{Z}/p\mathbb{Z})^{\ell+1}$, find $(x_1, \ldots, x_\ell) \in \{0,1\}^\ell$ s.t.

$$w_1 \cdot x_1 + \cdots + w_\ell \cdot x_\ell = t \bmod p$$

- **Linear relation:** $w_1 \cdot x_1 + \cdots + w_\ell \cdot x_\ell = t \bmod p$
- $x_i \in \{0,1\} \xrightarrow{\text{Arithmetization}} x_i(x_i - 1) = 0 \bmod p$
  $\rightsquigarrow \ell$ triples $\rightsquigarrow 2\ell$ auxiliary values + Tree PRG
- For $\ell = [\log_2(p)] = 256$, $n = 256 \rightsquigarrow 264$ KB!

| | |
|---|---|
| Shamir – Unpublished, 1986 | 1186 KB |
| Ling, Nguyen, Stehlé, Wang – PKC 2013 | 2350 KB |
| Beullens – Eurocrypt 2020 | 122 KB |
| Feneuil, Maire, Rivain, V. – Asiacrypt 2022 | 16 KB |